

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

HỆ THỐNG PHÂN LOẠI VÀ GỢI Ý
PHỎNG VẤN IT DỰA TRÊN
KNOWLEDGE TRACING
(InternHub KT Recommendation System)

Nhóm sinh viên thực hiện

Mai Dương Hải Đăng	24520253	24520253@gm.uit.edu.vn
Tạ Gia Khiêm	24520805	24520805@gm.uit.edu.vn
Nguyễn Sỹ Huỳnh	24520716	24520716@gm.uit.edu.vn
Nguyễn Ngọc Nam	24521114	24521114@gm.uit.edu.vn

Giáo viên hướng dẫn: TS. Nguyễn Văn Kiệt

TP. Hồ Chí Minh, 2026

Lời Cam Đoan

Tôi xin cam đoan khóa luận “*Hệ thống phân loại và gợi ý phỏng vấn IT dựa trên Knowledge Tracing (InternHub)*” là công trình nghiên cứu của riêng tôi dưới sự hướng dẫn của giáo viên hướng dẫn. Các kết quả, số liệu thực nghiệm trình bày trong khóa luận là trung thực và được tạo ra từ chính pipeline dữ liệu, mã nguồn và các thí nghiệm do tôi thực hiện.

Toàn bộ question bank được xây dựng từ các nguồn câu hỏi phỏng vấn công khai (kèm trích dẫn) kết hợp sinh tự động bằng mô hình ngôn ngữ lớn; dữ liệu huấn luyện Knowledge Tracing là dữ liệu **mô phỏng tổng hợp**, không sử dụng dữ liệu cá nhân của người dùng thật. Mọi tài liệu tham khảo đều được trích dẫn đầy đủ trong danh mục Tài liệu tham khảo.

Tôi xin chịu hoàn toàn trách nhiệm về tính trung thực của khóa luận.

TP. Hồ Chí Minh, 2026
Nhóm sinh viên thực hiện

Mai Dương Hải Đăng Tạ Gia Khiêm
Nguyễn Sỹ Huỳnh Nguyễn Ngọc Nam

Lời Cảm Ơn

Tôi xin gửi lời cảm ơn chân thành đến giáo viên hướng dẫn đã định hướng đề tài, góp ý về phương pháp luận và đồng hành trong suốt quá trình thực hiện khóa luận. Tôi cũng xin cảm ơn quý thầy cô Trường Đại học Công nghệ Thông tin – ĐHQG TP.HCM đã trang bị nền tảng kiến thức về Học máy, Khai phá dữ liệu và Kỹ thuật phần mềm, làm cơ sở cho công trình này.

Cuối cùng, tôi xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ trong thời gian thực hiện đề tài.

Tác giả khóa luận

Tóm Tắt

Khóa luận đề xuất và xây dựng *InternHub* – một hệ thống luyện phỏng vấn IT cá nhân hóa tích hợp **Knowledge Tracing (KT)** cho ba vai trò dữ liệu phổ biến: Data Analyst (DA), Data Scientist (DS) và Data Engineer (DE). Hệ thống giải quyết hai hạn chế của các nền tảng luyện phỏng vấn hiện nay: thiếu cá nhân hóa theo năng lực và không theo dõi được quỹ đạo kiến thức của người học.

Về dữ liệu, khóa luận xây dựng question bank đa dạng (5 loại câu hỏi, 17 Knowledge Components) bằng pipeline kết hợp thu thập từ các nguồn công khai và sinh tự động bằng LLM kèm kiểm định chất lượng; đồng thời thiết kế pipeline mô phỏng tương tác dựa trên Item Response Theory (IRT) để tạo bộ dữ liệu huấn luyện KT (1.000 người dùng ảo, ~40.000 tương tác). Về mô hình, khóa luận huấn luyện và so sánh ba họ KT – BKT, DKT, SAKT – cùng biến thể *quality label* liên tục. Kết quả trên tập kiểm tra cho thấy mọi mô hình KT vượt baseline (ROC-AUC ≈ 0.68 so với 0.50); biến thể **DKT-Quality** đạt PR-AUC và RMSE tốt nhất trong nhóm neural.

Về hệ thống, khóa luận thiết kế cơ chế cập nhật năng lực thời gian thực bằng cách kết hợp dự đoán KT (70%) và trung bình trượt mũ EMA (30%), tích hợp vào engine gợi ý theo chiến lược *zone of proximal development* và bài kiểm tra thích nghi hai giai đoạn (Multi-Stage Testing). Toàn bộ được hiện thực thành ứng dụng web hoàn chỉnh (React + FastAPI + Streamlit) với chấm điểm tự động bằng Gemini LLM và cơ chế chấm dự phòng cục bộ.

Từ khóa: Knowledge Tracing, Deep Knowledge Tracing, Item Response Theory, Hệ thống gợi ý, Computerized Adaptive Testing, Luyện phỏng vấn IT.

Abstract

This thesis proposes and implements *InternHub*, a personalized IT interview-practice platform powered by **Knowledge Tracing (KT)** for three popular data roles: Data Analyst, Data Scientist, and Data Engineer. It addresses two limitations of existing platforms: the lack of competency-based personalization and the inability to track a

learner’s knowledge trajectory. A diverse question bank (5 question types, 17 Knowledge Components) is built through a hybrid pipeline of public-source collection and LLM generation with quality control, while an IRT-based simulation pipeline produces the KT training data (1,000 virtual users, $\sim 40,000$ interactions). Three KT model families – BKT, DKT, SAKT – are trained and compared, including a continuous *quality-label* variant; all KT models beat the baseline (ROC-AUC ≈ 0.68 vs. 0.50), with DKT-Quality achieving the best PR-AUC and RMSE among neural models. A real-time skill-update mechanism blends KT prediction (70%) with an exponential moving average (30%), feeding a zone-of-proximal-development recommender and a two-stage adaptive test. The whole pipeline is realized as a complete web application (React + FastAPI + Streamlit) with automatic LLM-based grading and a local fallback grader.

Keywords: Knowledge Tracing, Deep Knowledge Tracing, Item Response Theory, Recommendation System, Computerized Adaptive Testing, IT Interview Preparation.

Contents

Lời Cam Đoan	i
Lời Cảm Ơn	ii
Tóm Tắt	iii
Danh Mục Từ Viết Tắt	xi
1 Giới Thiệu	2
1.1 Đặt Vấn Đề	2
1.2 Mục Tiêu Nghiên Cứu	3
1.3 Phạm Vi Nghiên Cứu	3
1.4 Đóng Góp Chính	4
1.5 Cấu Trúc Luận Văn	5
2 Cơ Sở Lý Thuyết	6
2.1 Knowledge Tracing	6
2.1.1 Bayesian Knowledge Tracing (BKT)	6
2.1.2 Deep Knowledge Tracing (DKT)	7
2.1.3 Self-Attentive Knowledge Tracing (SAKT)	7
2.2 Item Response Theory (IRT)	8
2.3 Recommendation System cho Interview Practice	8
2.4 Cập Nhật Năng Lực Thời Gian Thực: EMA và Blend KT-EMA	9
2.5 Các Công Trình Liên Quan	9
3 Xây Dựng Dữ Liệu và Phân Tích Khám Phá	11
3.1 Thu Thập và Kiểm Định Question Bank	11
3.1.1 Thu thập từ nguồn công khai	11
3.1.2 Sinh câu hỏi bằng LLM	11
3.1.3 Kiểm định chất lượng (QC)	12
3.1.4 Kết quả	12

3.2	Pipeline Xây Dựng Dữ Liệu	12
3.3	Phân Tích Question Bank	13
3.4	Phân Tích Người Dùng	14
3.5	Phân Tích Tương Tác	15
3.5.1	Phân phối quality score	15
3.5.2	Độ dài chuỗi tương tác	15
3.5.3	Quality theo mức độ khó	16
3.5.4	Quality heatmap theo competency	17
3.6	Phân Tích Sparsity và Learning Curve	18
3.6.1	Ma trận sparsity user-KC	18
3.6.2	Learning curve tổng thể	18
3.6.3	Learning curve theo loại người dùng	19
3.7	Phân Tích Confidence Bias	19
3.8	Tóm Tắt Đặc Tính Dữ Liệu	20
4	Thực Nghiệm và Đánh Giá	21
4.1	Môi Trường Thực Nghiệm	21
4.2	Bộ Dữ Liệu Mô Phỏng	21
4.2.1	Thống kê tổng quan	22
4.2.2	Kiểm tra tính toàn vẹn dữ liệu	22
4.3	Cấu Hình Thực Nghiệm (Ablation Study)	23
4.4	Độ Đo Đánh Giá	23
4.5	Kết Quả Ablation Study	24
4.6	Phân Tích Kết Quả	25
4.6.1	Hiệu quả chung của KT pipeline	25
4.6.2	Ảnh hưởng của quality label	25
4.6.3	BKT và simulation bias	26
4.6.4	Deep models và data size bottleneck	26
4.6.5	Hiệu quả tính toán	26
4.7	Phân Tích Sâu (Notebook 04)	26
4.7.1	Hiệu suất theo từng Knowledge Component	26
4.7.2	Phân tích theo vị trí trong chuỗi	27
4.7.3	Phân tích theo độ khó câu hỏi	27
4.7.4	Phân tích theo loại người dùng	28
4.7.5	Cold-start score và độ chính xác mô hình	28
4.7.6	Phân tích lỗi	29
4.8	Hạn Chế và Thảo Luận	29
4.8.1	Simulation bias (nghiêm trọng)	29

4.8.2	Class imbalance (trung bình)	29
4.8.3	KC frequency imbalance (nhẹ)	30
4.8.4	Data scale (nhẹ)	30
4.8.5	Sequence length hằng định (nhẹ)	30
4.8.6	Phân phối trình độ người dùng lệch (nhẹ)	30
4.9	Tóm Tắt Kết Quả	30
5	Thiết Kế và Hiện Thực Hệ Thống	32
5.1	Tổng Quan Kiến Trúc	32
5.2	Tầng Dữ Liệu và Engine Năng Lực	33
5.2.1	Chuẩn hóa câu hỏi	33
5.2.2	Ánh xạ KC (Competency Engine)	33
5.3	Tích Hợp Knowledge Tracing Thời Gian Thực	33
5.4	Engine Gợi Ý theo Zone of Proximal Development	34
5.5	Bài Kiểm Tra Thích Nghi (Multi-Stage Testing)	34
5.6	Hệ Thống Chấm Điểm	35
5.7	Tầng Phục Vụ và Giao Diện	35
5.7.1	Vì sao tách FastAPI khỏi Streamlit	35
5.7.2	Giao diện React không bundler	36
5.8	Minh Họa Hệ Thống	36
6	Kết Luận và Hướng Phát Triển	37
6.1	Tóm Tắt Những Gì Đã Thực Hiện	37
6.1.1	Xây dựng bộ dữ liệu	37
6.1.2	Huấn luyện và đánh giá mô hình KT	37
6.1.3	Hệ thống gợi ý và ứng dụng	38
6.2	Những Vấn Đề Đã Phát Hiện và Sửa Chữa	38
6.2.1	Simulation pipeline	38
6.2.2	Knowledge Tracing integration	39
6.2.3	API và Frontend integration	39
6.3	Hạn Chế Tổng Hợp	40
6.3.1	Hạn chế về dữ liệu	40
6.3.2	Hạn chế về mô hình	40
6.3.3	Hạn chế về hệ thống	40
6.4	Hướng Phát Triển Tương Lai	41
6.4.1	Dữ liệu thực và thực nghiệm người dùng	41
6.4.2	Cải tiến mô hình KT	41
6.4.3	Cải tiến hệ thống gợi ý	41

6.4.4	Cải tiến hệ thống chấm điểm	41
6.4.5	Production deployment	42
6.5	Kết Luận	42
A	Phụ Lục	43
A.1	Cấu Trúc Mã Nguồn	43
A.2	Prompt Chấm Điểm (Gemini)	43
A.3	Đặc Tả API (chi tiết)	44
A.4	Siêu Tham Số Mô Phỏng và Mô Hình	45
A.5	Năm Loại Câu Hỏi	45

List of Figures

3.1	Phân bố câu hỏi theo role và mức độ khó trong question bank	13
3.2	Độ phủ câu hỏi theo từng Knowledge Component (KC)	13
3.3	Phân bố người dùng theo role (trái) và trình độ (phải)	14
3.4	Heatmap skill vector của 50 người dùng mẫu (mỗi hàng là 1 user) . . .	14
3.5	Phân phối quality score (0 = Fail, 1 = Pass HR, 2 = Pass Tech)	15
3.6	Phân phối độ dài chuỗi tương tác mỗi người dùng	15
3.7	Phân phối quality score theo độ khó câu hỏi (EASY/MEDIUM/HARD)	16
3.8	Tỷ lệ Pass ($Q \geq 1$) trung bình theo competency và role	17
3.9	Ma trận tương tác user-KC: màu tối = có tương tác, trắng = không có	18
3.10	Learning curve: skill trung bình theo vị trí trong chuỗi tương tác . . .	18
3.11	Learning curve phân tầng theo trình độ người dùng (beginner/intermediate/advanced)	19
3.12	Self-rating so với skill thực: phát hiện confidence bias	19
4.1	So sánh ROC-AUC và PR-AUC của 8 cấu hình trên tập test	24
4.2	Learning curves của 6 mô hình neural (train loss, val loss, val AUC theo epoch)	25
4.3	ROC-AUC theo từng KC: BKT vs DKT-Quality★ trên tập test	26
4.4	ROC-AUC theo vị trí trong sequence: early (0–25%) vs late (75–100%)	27
4.5	ROC-AUC phân tầng theo độ khó: EASY / MEDIUM / HARD	27
4.6	ROC-AUC theo loại người dùng: beginner / intermediate / advanced .	28
4.7	Tương quan giữa cold-start score và per-user AUC (DKT-Quality★) . .	28
4.8	Confusion matrix và phân phối điểm dự đoán theo nhãn thực – DKT-Quality★	29
5.1	Kiến trúc ba tầng của <i>InternHub</i> : dữ liệu → KT/scoring → phục vụ. .	32
5.2	Luồng Multi-Stage Testing: Stage 1 định vị → phân nhánh → Stage 2 thích nghi.	34

List of Tables

3.1	Tóm tắt đặc tính bộ dữ liệu EDA	20
4.1	Thống kê bộ dữ liệu mô phỏng	22
4.2	Các cấu hình trong ablation study	23
4.3	Kết quả ablation study trên tập test (150 người dùng, 6,065 tương tác)	24
5.1	Các endpoint FastAPI của <i>InternHub</i>	36
A.1	Tham số pipeline mô phỏng dữ liệu	45
A.2	Hàng số runtime của hệ thống gợi ý	45
A.3	Các loại câu hỏi trong question bank	45

Danh Mục Từ Viết Tắt

Từ viết tắt	Diễn giải
KT	Knowledge Tracing – Theo dõi tri thức
BKT	Bayesian Knowledge Tracing
DKT	Deep Knowledge Tracing
SAKT	Self-Attentive Knowledge Tracing
IRT	Item Response Theory – Lý thuyết ứng đáp câu hỏi
CAT	Computerized Adaptive Testing – Kiểm tra thích nghi
MST	Multi-Stage Testing – Kiểm tra nhiều giai đoạn
KC	Knowledge Component – Thành phần tri thức
EMA	Exponential Moving Average – Trung bình trượt mũ
ZPD	Zone of Proximal Development – Vùng phát triển gần
LLM	Large Language Model – Mô hình ngôn ngữ lớn
ROC-AUC	Area Under the ROC Curve
PR-AUC	Area Under the Precision-Recall Curve
RMSE	Root Mean Squared Error
LSTM	Long Short-Term Memory
SPA	Single-Page Application
API	Application Programming Interface
EDA	Exploratory Data Analysis – Phân tích khám phá dữ liệu
QC	Quality Control – Kiểm định chất lượng
DA / DS / DE	Data Analyst / Data Scientist / Data Engineer

17 Knowledge Components (KC) theo vai trò:

Vai trò	Mã KC	Ý nghĩa
	SQL_DATABASE	SQL & cơ sở dữ liệu
	BI_VISUALIZATION	Trực quan hóa & BI
DA		

Vai trò	Mã KC	Ý nghĩa
	STATISTICS_EXPERIMENTATION	Thống kê & thực nghiệm
	ANALYTICS_BUSINESS	Phân tích nghiệp vụ
	PYTHON_ANALYTICS	Python cho phân tích
DS	ALGORITHM_THEORY	Thuật toán & lý thuyết ML
	EVALUATION_METRICS	Độ đo đánh giá mô hình
	DATA_PREPROCESSING	Tiền xử lý dữ liệu
	DEEP_LEARNING	Học sâu
	NLP	Xử lý ngôn ngữ tự nhiên
	TIME_SERIES	Chuỗi thời gian
	MLOPS	Vận hành ML (MLOps)
DE	DATA_PIPELINE	Đường ống dữ liệu (ETL/ELT)
	DATA_ARCHITECTURE_MODELING	Kiến trúc & mô hình hóa dữ liệu
	BIG_DATA_CLOUD_TOOLS	Big data & công cụ cloud
	DATABASE_INTERNALS	Cơ chế nội tại CSDL
	SYSTEM_ARCHITECTURE	Kiến trúc hệ thống

Chapter 1

Giới Thiệu

1.1 Đặt Vấn Đề

Thị trường tuyển dụng nhân lực công nghệ thông tin tại Việt Nam và trên thế giới đang chứng kiến sự cạnh tranh ngày càng gay gắt. Theo khảo sát của TopDev (2024) **topdev2024**, hơn 70% ứng viên IT tại Việt Nam thừa nhận chưa chuẩn bị đủ kỹ năng kỹ thuật cần thiết trước các vòng phỏng vấn, trong khi phần lớn các nền tảng luyện phỏng vấn hiện có (LeetCode, HackerRank, Pramp) chủ yếu tập trung vào lập trình thuật toán, thiếu nội dung dành riêng cho ba vai trò dữ liệu phổ biến nhất: Data Analyst (DA), Data Scientist (DS) và Data Engineer (DE).

Hai thách thức cốt lõi tồn tại với các nền tảng luyện phỏng vấn hiện tại:

(1) **Thiếu cá nhân hóa theo năng lực.** Hầu hết hệ thống cung cấp câu hỏi theo cách tĩnh (static), không phản ánh được trình độ thực tế của từng ứng viên. Người dùng có kinh nghiệm phải làm lại các câu hỏi quá dễ, trong khi người mới bắt đầu có thể nản lòng với câu hỏi ngoài tầm. Vấn đề này tương ứng với khái niệm *zone of proximal development* trong tâm lý học giáo dục **vygotsky1978mind** – câu hỏi hiệu quả nhất là câu hỏi hơi khó hơn năng lực hiện tại của người học một chút.

(2) **Không theo dõi được quỹ đạo kiến thức.** Hệ thống hiện có không duy trì và cập nhật trạng thái kiến thức (knowledge state) của người học theo thời gian. Do đó, sau mỗi phiên luyện tập, hệ thống không biết người dùng đã tiến bộ ở điểm nào, còn yếu ở kỹ năng nào, và câu hỏi tiếp theo nên theo hướng nào.

Knowledge Tracing (KT) **corbett1994knowledge** là hướng giải quyết được nghiên cứu nhiều trong lĩnh vực Educational Data Mining, giúp mô hình hóa và dự đoán trạng thái kiến thức của người học qua lịch sử tương tác. Các mô hình KT từ Bayesian Knowledge Tracing **corbett1994knowledge** đến Deep Knowledge Tracing **piech2015deep** và Self-Attentive Knowledge Tracing **pandey2019self** đã chứng minh hiệu quả trong môi trường học thuật (MOOC, bài tập toán, ngôn ngữ). Tuy nhiên, việc áp dụng KT

cho bài toán luyện phỏng vấn IT – với đặc thù câu hỏi mở, chấm điểm tự động bằng LLM và nhiều loại câu hỏi (lý thuyết, coding, tình huống) – vẫn là một bài toán mở.

Luận văn này đề xuất và xây dựng *InternHub*, một hệ thống luyện phỏng vấn IT tích hợp KT, nhằm giải quyết đồng thời hai thách thức trên: theo dõi năng lực tự động và gợi ý câu hỏi cá nhân hóa theo từng vai trò nghề nghiệp.

1.2 Mục Tiêu Nghiên Cứu

Luận văn đặt ra bốn mục tiêu chính:

1. **Xây dựng bộ dữ liệu luyện KT phù hợp cho bài toán phỏng vấn IT.** Do thiếu dữ liệu thực, luận văn thiết kế pipeline mô phỏng dữ liệu tổng hợp có tính chất thực tế cao, kết hợp IRT **embretson2000item** để sinh hành vi người dùng và LLM để xây dựng question bank đa dạng cho ba vai trò DA/DS/DE.
2. **Huấn luyện và so sánh các mô hình KT.** Triển khai và đánh giá ba họ mô hình – BKT **corbett1994knowledge**, DKT **piech2015deep**, SAKT **pandey2019self** – với các biến thể sử dụng nhãn nhị phân và quality label liên tục. Phân tích điểm mạnh/yếu của từng mô hình trên tập dữ liệu phỏng vấn IT.
3. **Xây dựng engine gợi ý dựa trên KT.** Thiết kế cơ chế cập nhật skill vector bằng cách kết hợp KT prediction (70%) và EMA (30%), tích hợp vào pipeline đề xuất câu hỏi theo chiến lược zone of proximal development, có hỗ trợ Computerized Adaptive Testing (CAT) hai giai đoạn **wainer2000computerized**.
4. **Tích hợp vào ứng dụng web hoàn chỉnh.** Xây dựng *InternHub* – giao diện React SPA được phục vụ qua Streamlit, kết nối với FastAPI backend – hỗ trợ chấm điểm tự động bằng Gemini LLM và fallback local rubric **team2023gemini**.

1.3 Phạm Vi Nghiên Cứu

Trong phạm vi:

- Ba vai trò nghề nghiệp: Data Analyst (DA), Data Scientist (DS), Data Engineer (DE).
- 17 Knowledge Components (KC) phân bổ theo vai trò: DA (5 KC), DS (7 KC), DE (5 KC).

- Bộ câu hỏi gồm 5 loại: trắc nghiệm (MC_SINGLE), đúng/sai (TRUE_FALSE), điền chỗ trống (FILL_BLANK), lý thuyết mở (THEORY), và coding (CODING/CODING_EXERCISE).
- Dữ liệu training KT: 1,000 người dùng mô phỏng, $\sim 40,000$ tương tác.
- Chấm điểm câu hỏi mở bằng Gemini 3.5 Flash với local rubric fallback.

Ngoài phạm vi:

- Thu thập và sử dụng dữ liệu người dùng thực (do giới hạn thời gian và quyền riêng tư).
- Triển khai production với load balancing và autoscaling.
- Đánh giá A/B testing so với các hệ thống thương mại.
- Hỗ trợ các ngôn ngữ lập trình thực thi trong sandbox (chỉ hiển thị).

1.4 Đóng Góp Chính

Luận văn có bốn đóng góp chính:

1. **Pipeline xây dựng dữ liệu phỏng vấn IT tổng hợp:** Một pipeline đầy đủ từ sinh câu hỏi (LLM), tạo người dùng ảo (correlated Gaussian skill), đến mô phỏng tương tác (IRT + power-based graded response). Pipeline này có thể tái sử dụng cho các bài toán KT trong lĩnh vực nghề nghiệp khác.
2. **So sánh hệ thống KT với quality label:** Đề xuất và đánh giá biến thể DKT-Quality sử dụng điểm chất lượng liên tục $\in \{0, 0.5, 1\}$ thay vì nhãn nhị phân, cho thấy cải thiện đáng kể về RMSE và PR-AUC trên bộ dữ liệu phỏng vấn IT.
3. **Cơ chế tích hợp KT-EMA cho real-time skill update:** Công thức blend KT(70%) + EMA(30%) với ngưỡng kích hoạt tối thiểu 3 tương tác, giải quyết bài toán cold-start và đảm bảo tính ổn định trong giai đoạn đầu.
4. **Hệ thống luyện phỏng vấn IT end-to-end:** Ứng dụng *InternHub* hoàn chỉnh tích hợp KT, 2-Stage Branching CAT, chấm điểm LLM, và giao diện React với FastAPI backend – tất cả trong một codebase duy nhất có thể chạy local.

1.5 Cấu Trúc Luận Văn

Phần còn lại của luận văn được tổ chức như sau:

Chương 2 – Cơ Sở Lý Thuyết: Trình bày nền tảng lý thuyết của Knowledge Tracing (BKT, DKT, SAKT), Item Response Theory, và hệ thống gợi ý trong ngữ cảnh luyện phỏng vấn. Đồng thời khảo sát các công trình liên quan trong lĩnh vực.

Chương 3 – Xây Dựng Dữ Liệu và Phân Tích Khám Phá: Mô tả chi tiết pipeline xây dựng question bank và dữ liệu mô phỏng, kết hợp với phân tích EDA toàn diện về đặc tính bộ dữ liệu.

Chương 4 – Thực Nghiệm và Đánh Giá: Trình bày thiết kế thực nghiệm, kết quả huấn luyện và đánh giá các mô hình KT, phân tích so sánh, và thảo luận về hạn chế.

Chương 6 – Kết Luận và Hướng Phát Triển: Tổng kết kết quả đạt được, đóng góp, và đề xuất hướng phát triển cho nghiên cứu tương lai.

Chapter 2

Cơ Sở Lý Thuyết

2.1 Knowledge Tracing

Knowledge Tracing (KT) là bài toán theo dõi và dự đoán trạng thái kiến thức của người học qua thời gian, dựa trên lịch sử tương tác với các bài tập. Cho một chuỗi tương tác $\{(q_1, r_1), (q_2, r_2), \dots, (q_t, r_t)\}$ trong đó q_t là câu hỏi tại thời điểm t và $r_t \in \{0, 1\}$ là kết quả (đúng/sai), mục tiêu của KT là dự đoán xác suất trả lời đúng câu hỏi tiếp theo q_{t+1} :

$$p(r_{t+1} = 1 \mid q_{t+1}, q_1, r_1, \dots, q_t, r_t) \quad (2.1)$$

Trong hệ thống *InternHub*, bài toán KT được áp dụng để theo dõi năng lực của ứng viên trên 17 nhóm kỹ năng (Knowledge Components – KC) thuộc ba vai trò nghề nghiệp IT: Data Analyst (DA), Data Scientist (DS) và Data Engineer (DE).

2.1.1 Bayesian Knowledge Tracing (BKT)

BKT **corbett1994knowledge** mô hình hóa từng KC như một chuỗi Markov ẩn (Hidden Markov Model – HMM) với hai trạng thái: *chưa thành thạo* ($L = 0$) và *thành thạo* ($L = 1$). Mô hình được đặc trưng bởi bốn tham số cho mỗi KC:

- $P(L_0)$: xác suất ban đầu người học đã thành thạo KC.
- $P(T)$: xác suất chuyển từ chưa thành thạo sang thành thạo sau mỗi lần thực hành.
- $P(S)$: xác suất trả lời sai dù đã thành thạo (*slip*).
- $P(G)$: xác suất trả lời đúng dù chưa thành thạo (*guess*).

Xác suất trả lời đúng tại bước t được tính:

$$P(r_t = 1) = P(L_t)(1 - P(S)) + (1 - P(L_t))P(G) \quad (2.2)$$

Sau mỗi quan sát, trạng thái thành thạo được cập nhật theo Bayes:

$$P(L_t | r_t = 1) = \frac{P(L_t)(1 - P(S))}{P(r_t = 1)} \quad (2.3)$$

$$P(L_{t+1}) = P(L_t | r_t) + (1 - P(L_t | r_t)) \cdot P(T) \quad (2.4)$$

Tham số của BKT được ước lượng bằng thuật toán Expectation-Maximization (EM) hoặc tối ưu hóa trực tiếp L-BFGS-B trên log-likelihood. Ưu điểm của BKT là khả năng diễn giải cao và hiệu quả trên dữ liệu nhỏ; hạn chế là giả định độc lập giữa các KC và không mô hình hóa được tương tác bài tập phức tạp.

2.1.2 Deep Knowledge Tracing (DKT)

DKT [piech2015deep](#) thay thế mô hình xác suất của BKT bằng mạng nơ-ron hồi quy (LSTM), cho phép học được biểu diễn ẩn phong phú hơn cho trạng thái kiến thức của người học. Đầu vào tại mỗi bước t là vector one-hot mã hóa cặp (q_t, r_t) , có chiều $2 \cdot |\mathcal{Q}|$ với $|\mathcal{Q}|$ là số lượng câu hỏi (hoặc KC).

$$\mathbf{h}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}) \quad (2.5)$$

$$\hat{y}_{t+1} = \sigma(\mathbf{W} \cdot \mathbf{h}_t + \mathbf{b}) \quad (2.6)$$

trong đó $\hat{y}_{t+1}$ là xác suất trả lời đúng câu hỏi tiếp theo, σ là hàm sigmoid.

Trong *InternHub*, DKT được mở rộng với **quality label**: thay vì nhãn nhị phân $r_t \in \{0, 1\}$, mô hình sử dụng điểm chất lượng liên tục $r_t \in [0, 1]$ phản ánh mức độ thành thạo (điểm tự đánh giá sau phỏng vấn). Hàm mất mát chuyển từ Binary Cross-Entropy sang Mean Squared Error:

$$\mathcal{L}_{\text{quality}} = \frac{1}{T} \sum_{t=1}^T (r_t - \hat{y}_t)^2 \quad (2.7)$$

2.1.3 Self-Attentive Knowledge Tracing (SAKT)

SAKT [pandey2019self](#) thay thế kiến trúc hồi quy của DKT bằng cơ chế self-attention, cho phép mô hình học được mối liên hệ trực tiếp giữa các KC liên quan trong lịch sử tương tác.

Cho chuỗi tương tác $\{(q_1, r_1), \dots, (q_{t-1}, r_{t-1})\}$, SAKT xây dựng:

- **Exercise embedding:** $\mathbf{e}_{q_t} \in \mathbb{R}^d$ cho câu hỏi cần dự đoán.
- **Interaction embedding:** $\mathbf{m}_s = \mathbf{e}_{q_s} + \mathbf{r}_s$ cho mỗi bước lịch sử.

Attention score giữa câu hỏi q_t và bước lịch sử s được tính:

$$a_{t,s} = \frac{(\mathbf{e}_{q_t} \mathbf{W}^Q)(\mathbf{m}_s \mathbf{W}^K)^\top}{\sqrt{d}} \quad (2.8)$$

$$\hat{y}_t = \sigma \left(\mathbf{W}_o \cdot \sum_s \text{softmax}(a_{t,s}) \cdot \mathbf{m}_s \mathbf{W}^V \right) \quad (2.9)$$

Ưu điểm của SAKT là khả năng song song hóa (không sequential như LSTM) và diễn giải được attention weight để hiểu KC nào ảnh hưởng đến dự đoán.

2.2 Item Response Theory (IRT)

Item Response Theory **embretson2000item** là khung lý thuyết mô hình hóa mối quan hệ giữa năng lực người học θ và xác suất trả lời đúng câu hỏi i . Mô hình 3PL (3-Parameter Logistic) được dùng trong simulation:

$$P(r = 1 \mid \theta, a_i, b_i, c_i) = c_i + (1 - c_i) \cdot \frac{1}{1 + e^{-a_i(\theta - b_i)}} \quad (2.10)$$

trong đó a_i là độ phân biệt (discrimination), b_i là độ khó (difficulty), c_i là xác suất đoán ngẫu nhiên (guessing). Trong *InternHub*, tham số IRT được ánh xạ từ cấu hình độ khó EASY/MEDIUM/HARD với $b \in \{0.30, 0.55, 0.80\}$.

2.3 Recommendation System cho Interview Practice

Hệ thống gợi ý trong *InternHub* kết hợp KT và Collaborative Filtering để đề xuất câu hỏi phù hợp với trình độ hiện tại của ứng viên. Cụ thể, sau mỗi phiên luyện tập, vector năng lực $\boldsymbol{\theta}_u \in \mathbb{R}^{17}$ của người dùng u (17 KC) được cập nhật bởi KT model, sau đó được dùng để xếp hạng câu hỏi theo chiến lược *zone of proximal development*: ưu tiên câu hỏi thuộc KC yếu và độ khó phù hợp.

2.4 Cập Nhật Năng Lực Thời Gian Thực: EMA và Blend KT–EMA

Mô hình KT cho dự đoán tốt khi đã có đủ lịch sử tương tác, nhưng kém ổn định ở giai đoạn đầu (*cold-start*) khi người dùng mới chỉ trả lời vài câu. Để giải quyết, *InternHub* kết hợp KT với **Exponential Moving Average (EMA)** – một bộ ước lượng trực tuyến đơn giản, ổn định và không cần lịch sử dài.

Với mỗi KC, sau khi quan sát điểm số $s_t \in [0, 1]$ tại bước t , giá trị EMA được cập nhật:

$$\theta_t^{\text{EMA}} = \alpha s_t + (1 - \alpha) \theta_{t-1}^{\text{EMA}}, \quad \alpha = 0.35. \quad (2.11)$$

EMA luôn chạy ngay từ tương tác đầu tiên, cung cấp ước lượng năng lực thô. Khi lịch sử đạt tối thiểu $H_{\min} = 3$ tương tác, dự đoán KT được kích hoạt và **blend** với EMA theo trọng số cố định:

$$\hat{\theta}_t = \lambda \theta_t^{\text{KT}} + (1 - \lambda) \theta_t^{\text{EMA}}, \quad \lambda = 0.70. \quad (2.12)$$

Công thức (2.12) là một trong các đóng góp chính của khóa luận. Lựa chọn $\lambda = 0.7$ cho KT trọng số trội (vì KT nắm bắt được động lực chuỗi và quan hệ giữa các KC), trong khi giữ 0.3 cho EMA nhằm *ổn định hóa* dự đoán, chống dao động mạnh khi KT gặp chuỗi nhiễu. Ngưỡng $H_{\min} = 3$ đảm bảo KT chỉ tham gia khi đã có đủ ngữ cảnh, còn trước đó hệ thống dùng EMA thuần. Hằng số λ và $H_{\min}$ được giữ **nhất quán** giữa backend Python và frontend để bảo đảm con số hiển thị đồng bộ trên toàn hệ thống (xem Chương 5).

2.5 Các Công Trình Liên Quan

Knowledge Tracing. Từ BKT [corbett1994knowledge](#), các mô hình KT đã phát triển qua DKT [piech2015deep](#), DKVMN [zhang2017dynamic](#), AKT [ghosh2020context](#), và gần đây là các mô hình Transformer-based như SAINT [choi2020towards](#). SAKT [pandey2019self](#) là mô hình attention đầu tiên đơn giản và hiệu quả trong bài toán này.

Adaptive Learning Systems. Knewton [ferreira2017knewton](#) và các hệ thống ITS (Intelligent Tutoring System) như ASSISTments [heffernan2014assistments](#) sử dụng KT để cá nhân hóa lộ trình học. Tuy nhiên, các hệ thống này tập trung vào môi trường học thuật, chưa có ứng dụng cụ thể cho phổ thông IT.

Interview Preparation Systems. Các nền tảng như LeetCode, HackerRank cung cấp bài tập lập trình nhưng thiếu cơ chế theo dõi kiến thức có cấu trúc. *InternHub*

khác biệt ở chỗ tích hợp KT với question bank đa dạng (lý thuyết, thực hành, coding) cho ba vai trò DA/DS/DE, đồng thời sử dụng LLM để chấm điểm tự động.

Chapter 3

Xây Dựng Dữ Liệu và Phân Tích Khám Phá

Chương này mô tả toàn bộ pipeline xây dựng dữ liệu mô phỏng và phân tích khám phá (EDA) nhằm hiểu rõ đặc tính của bộ dữ liệu trước khi huấn luyện mô hình Knowledge Tracing.

3.1 Thu Thập và Kiểm Định Question Bank

Khác với phần lớn nghiên cứu KT chỉ dùng dataset có sẵn, khóa luận tự xây dựng question bank phỏng vấn IT qua một quy trình lai (hybrid) gồm bốn giai đoạn: thu thập, sinh tự động, kiểm định chất lượng, và chuẩn hóa.

3.1.1 Thu thập từ nguồn công khai

Câu hỏi được thu thập (scraping) từ nhiều kho câu hỏi phỏng vấn mở, sau đó trích dẫn nguồn: bộ câu hỏi Data Engineer (Obenner), bộ câu hỏi SQL (LearningZone), và các bộ câu hỏi DS/DA cộng đồng (Youssef, Alexey). Mỗi câu được ánh xạ sơ bộ về vai trò (DA/DS/DE) và nhóm kỹ năng (`skill_groups`).

3.1.2 Sinh câu hỏi bằng LLM

Để tăng độ phủ các KC còn thiếu, một pipeline đa mô hình được sử dụng: **Gemini Flash** sinh câu hỏi mới theo từng KC và mức độ; mô hình mạnh hơn tạo *reference answer* và *evaluation points*; một mô hình *judge* chấm chất lượng và một mô hình *enhancer* tinh chỉnh diễn đạt. Bốn loại câu hỏi khách quan (MC_SINGLE, TRUE_FALSE, FILL_BLANK) và hai loại tự luận (THEORY, CODING) được sinh kèm metadata đầy đủ.

3.1.3 Kiểm định chất lượng (QC)

Mỗi câu hỏi đi qua: (1) **kiểm tra schema** theo `question_bank.schema.json` (bắt buộc các trường `question_id`, `question_text`, `roles`, `difficulty_label`, `skill_groups`, `answers`); (2) **QC bằng RAG** đối chiếu nội dung với tài liệu tham chiếu để phát hiện câu sai sự thật; (3) **human review** trên mẫu ngẫu nhiên; và (4) **chuẩn hóa skill_groups** – ánh xạ các tag chi tiết (ví dụ `SQL_WINDOW_FUNCTION`) về 17 KC chuẩn qua bảng tra `SKILL_GROUP_TO_KC` (82 ánh xạ).

3.1.4 Kết quả

Snapshot dùng cho thực nghiệm KT gồm **1.578 câu** (17 KC, 3 mức độ). Sau khi tiếp tục thu thập và sinh bổ sung, question bank của ứng dụng triển khai mở rộng lên **2.122 câu**. Quá trình QC cũng phát hiện và xử lý một số lỗi dữ liệu điển hình (ví dụ câu bị tách rời tiêu đề khỏi nội dung khi scraping), minh họa tầm quan trọng của bước kiểm định.

3.2 Pipeline Xây Dựng Dữ Liệu

Do thiếu dữ liệu thực từ người dùng thật, luận văn sử dụng pipeline mô phỏng ba bước để tạo bộ dữ liệu training:

1. **Xây dựng question bank:** Thu thập và QC câu hỏi phỏng vấn qua pipeline 3 mô hình LLM (Gemini Flash sinh câu hỏi, GPT-4o làm reference answer, Claude Haiku judge chất lượng, Claude Sonnet enhance). Kết quả của snapshot dùng cho thực nghiệm KT: 1,578 câu hỏi unique, phân loại theo 3 role (DA/DS/DE), 3 mức độ (EASY/MEDIUM/HARD) và 17 Knowledge Components (KC). Về sau, question bank được mở rộng lên 2,122 câu cho ứng dụng triển khai (xem Chương 5).
2. **Tạo virtual users:** Sinh 1,000 người dùng ảo với skill vector theo phân phối correlated Gaussian, phản ánh tương quan thực tế giữa các competency (ví dụ: DA có $r(\text{SQL}, \text{Analytics}) \approx 0.55$; DS có $r(\text{Algorithm}, \text{Evaluation}) \approx 0.70$).
3. **Mô phỏng interaction:** Với mỗi người dùng, sinh chuỗi tương tác dựa trên:
 - **IRT:** $P(\text{correct}) = \sigma((\theta - b) \times 6)$, trong đó θ là skill hiện tại và b là độ khó câu hỏi.
 - **Power-based graded response:** $P(Q = 2) = p^\alpha$, $P(Q = 0) = (1 - p)^\alpha$, $\alpha = 2.0$.

- **Skill update:** $\theta += \eta(1 - \theta)$ khi đúng; $\theta -= \gamma\theta$ khi sai, với $\eta = 0.08$, $\gamma = 0.002$.

3.3 Phân Tích Question Bank

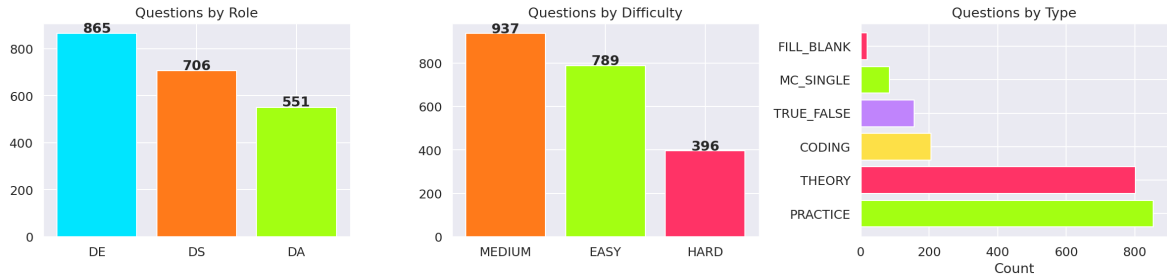


Figure 3.1: Phân bố câu hỏi theo role và mức độ khó trong question bank

Hình 3.1 cho thấy question bank có phân bố khá đồng đều giữa ba role (DA/DS/DE), với mỗi role có khoảng 500–550 câu hỏi. Trong mỗi role, câu hỏi MEDIUM chiếm tỷ lệ lớn nhất, phù hợp với thực tế phỏng vấn (câu hỏi quá dễ hoặc quá khó đều ít giá trị phân biệt).

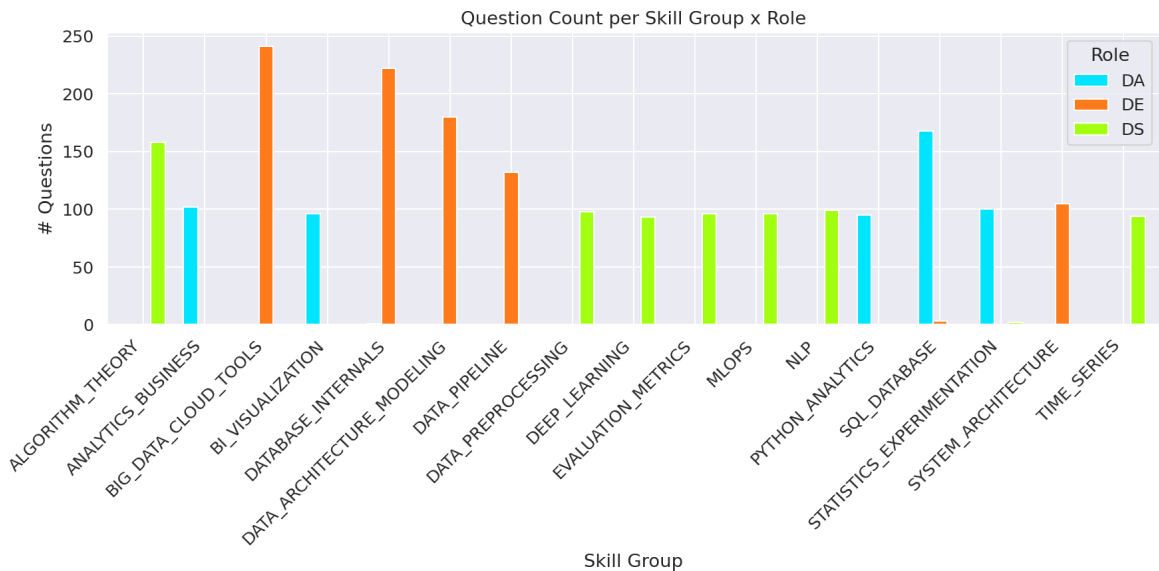


Figure 3.2: Độ phủ câu hỏi theo từng Knowledge Component (KC)

Hình 3.2 thể hiện số lượng câu hỏi theo từng KC. Có sự chênh lệch rõ rệt: một số KC như SQL_DATABASE (DA) và ALGORITHM_THEORY (DS) có số câu hỏi nhiều hơn, phản ánh tầm quan trọng và độ phổ biến của các chủ đề này trong thực tế phỏng vấn IT.

3.4 Phân Tích Người Dùng

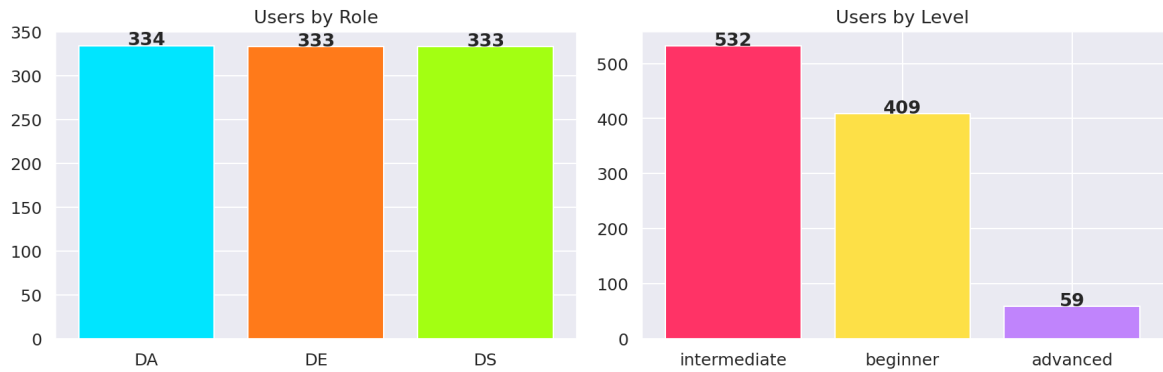


Figure 3.3: Phân bố người dùng theo role (trái) và trình độ (phải)

Hình 3.3 (trái) xác nhận phân bố đồng đều theo role: DA = 334, DS = 333, DE = 333. Biểu đồ phải cho thấy phân phối trình độ thực tế sau khi phân loại dựa trên skill vector: intermediate = 532 (53.2%), beginner = 409 (40.9%), advanced = 59 (5.9%). Phân phối này lệch so với mục tiêu thiết kế (35%/50%/15%) do quá trình phân loại lại (*reclassification*) dựa trên skill vector thực tế sau sampling – nhiều người dùng thuộc tier advanced bị reclassify về intermediate khi mean skill nằm trong khoảng $[0.40, 0.70]$.

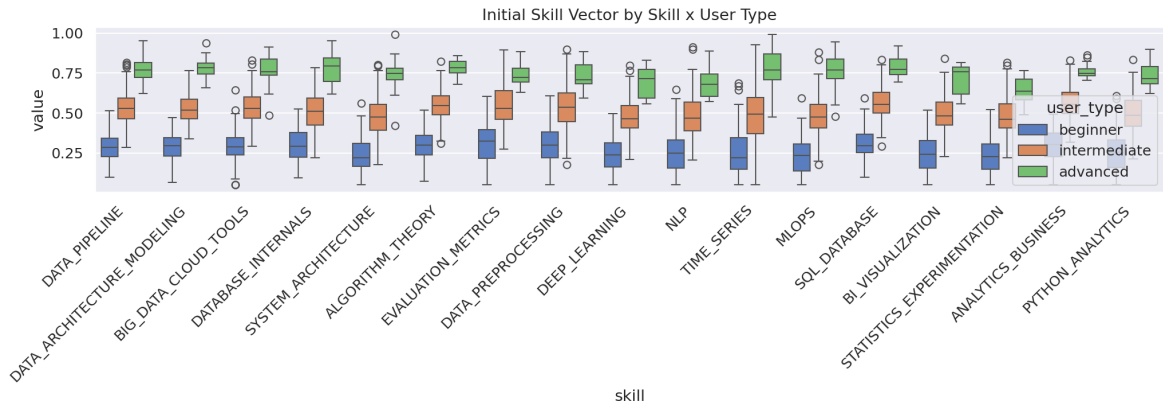


Figure 3.4: Heatmap skill vector của 50 người dùng mẫu (mỗi hàng là 1 user)

Hình 3.4 minh họa sự đa dạng của skill vector giữa các người dùng. Mỗi hàng đại diện cho một người dùng; màu sắc phản ánh mức skill từ thấp (xanh lam) đến cao (vàng/đỏ). Có thể quan sát tương quan dương giữa các KC trong cùng role – người dùng có skill cao ở một KC thường có skill trung bình trở lên ở các KC liên quan.

3.5 Phân Tích Tương Tác

3.5.1 Phân phối quality score

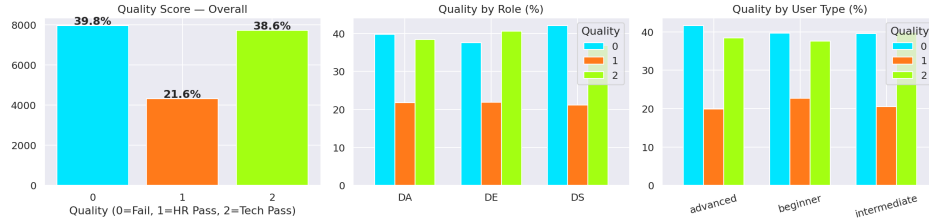


Figure 3.5: Phân phối quality score (0 = Fail, 1 = Pass HR, 2 = Pass Tech)

Hình 3.5 cho thấy phân phối quality score với Q=2 chiếm tỷ lệ cao nhất ($\sim 62\%$), Q=1 khoảng 20%, và Q=0 ($\sim 18\%$). Phân phối này phản ánh mô hình power-based graded response với $\alpha = 2.0$: bucket Q=1 (partial credit) rộng nhất ở vùng skill trung bình, trong khi Q=2 và Q=0 tập trung ở hai đầu phổ skill.

3.5.2 Độ dài chuỗi tương tác

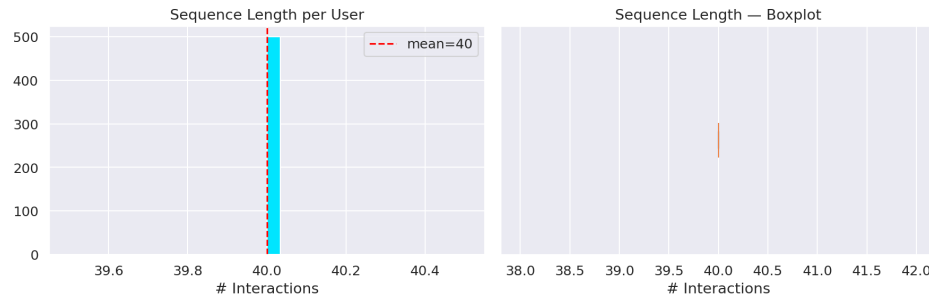


Figure 3.6: Phân phối độ dài chuỗi tương tác mỗi người dùng

Hình 3.6 cho thấy toàn bộ người dùng có đúng 40 tương tác. Mặc dù pipeline thiết kế randomize 20–60 câu/user, trên thực tế mỗi role chỉ có ~ 40 câu trong pool sau bước stratified sampling theo competency, dẫn đến sequence length bị cap tại pool size. Đây là hạn chế của quy mô question bank hiện tại (xem Mục 4.8 để thảo luận chi tiết).

3.5.3 Quality theo mức độ khó

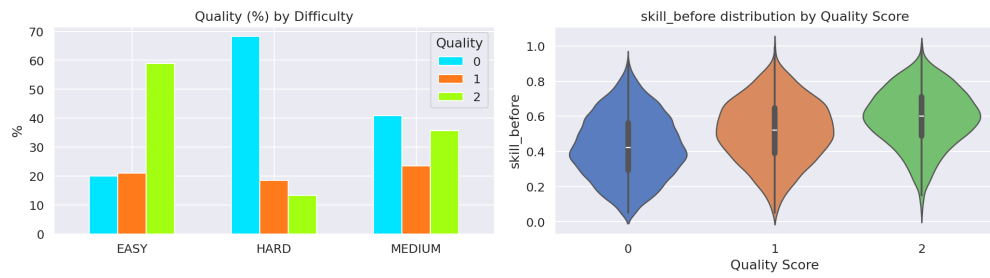


Figure 3.7: Phân phối quality score theo độ khó câu hỏi (EASY/MEDIUM/HARD)

Hình 3.7 xác nhận tính nhất quán của mô hình IRT: câu hỏi EASY có tỷ lệ $Q=2$ cao hơn rõ rệt, câu HARD có tỷ lệ $Q=0$ cao hơn. Câu hỏi MEDIUM có phân phối cân bằng nhất giữa ba mức – đây là vùng có độ phân biệt (discrimination) cao nhất trong IRT, phù hợp nhất cho training.

3.5.4 Quality heatmap theo competency

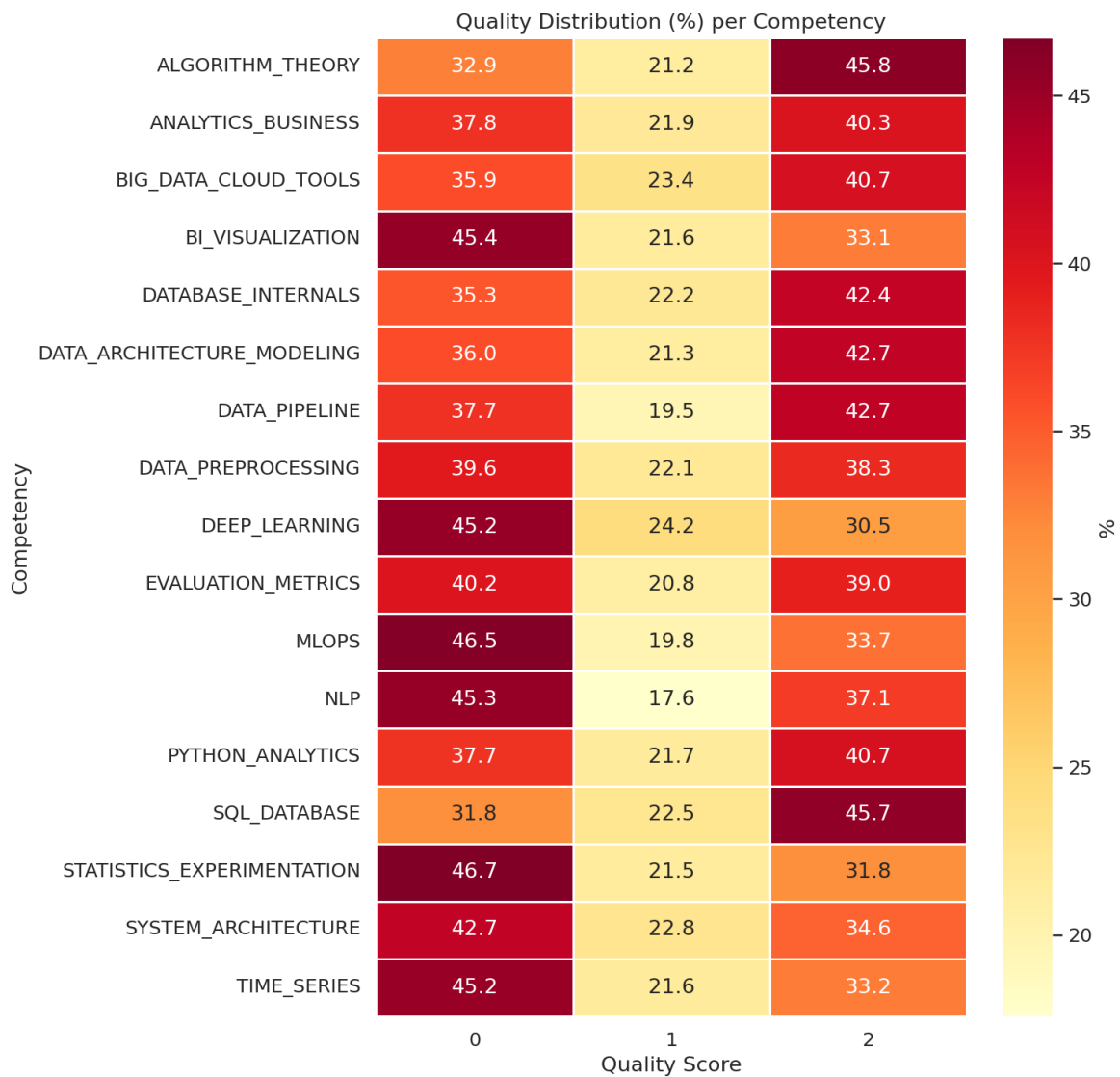


Figure 3.8: Tỷ lệ Pass ($Q \geq 1$) trung bình theo competency và role

Hình 3.8 cho thấy tỷ lệ pass khác nhau đáng kể giữa các competency. Trong nhóm DA, SQL_DATABASE có pass rate cao hơn STATISTICS_EXPERIMENTATION – phản ánh penalty mà pipeline thiết kế (người DA thường yếu statistics hơn SQL). Các KC của DS như DEEP_LEARNING và NLP có pass rate thấp nhất, nhất quán với độ phức tạp cao của các chủ đề này.

3.6 Phân Tích Sparsity và Learning Curve

3.6.1 Ma trận sparsity user–KC

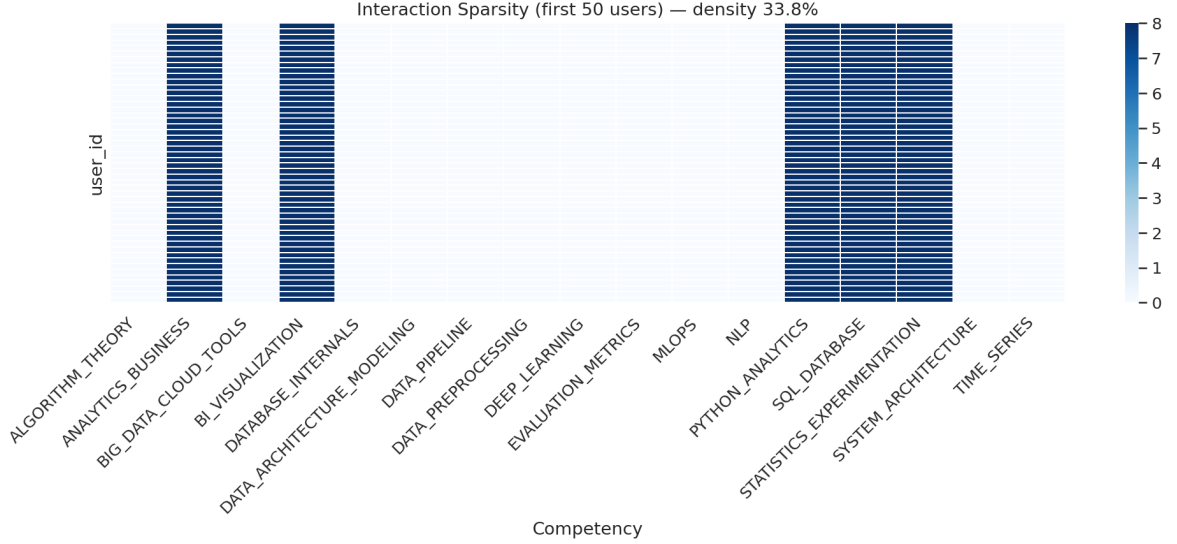


Figure 3.9: Ma trận tương tác user–KC: màu tối = có tương tác, trắng = không có

Hình 3.9 minh họa ma trận user–KC. Do mỗi user chỉ trả lời câu hỏi thuộc role của mình, matrix có cấu trúc block diagonal rõ ràng: DA users chỉ có tương tác với KC của DA, tương tự cho DS và DE. Điều này dẫn đến sparsity cao khi xét toàn bộ không gian KC (~77% entries rỗng), nhưng sparsity trong từng role thấp hơn đáng kể (~15–20%), đủ để BKT hoạt động ổn định.

3.6.2 Learning curve tổng thể

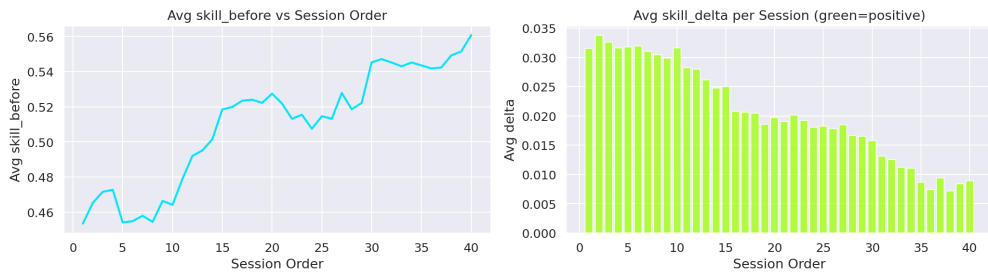


Figure 3.10: Learning curve: skill trung bình theo vị trí trong chuỗi tương tác

Hình 3.10 xác nhận rằng pipeline mô phỏng có learning dynamics đúng: skill trung bình tăng dần theo thứ tự câu hỏi trong session. Xu hướng tăng phản ánh tham số học $\eta = 0.08$ và forgetting rate thấp $\gamma = 0.002$, tạo ra learning curve có hình dạng đặc trưng của mô hình exponential growth with saturation.

3.6.3 Learning curve theo loại người dùng

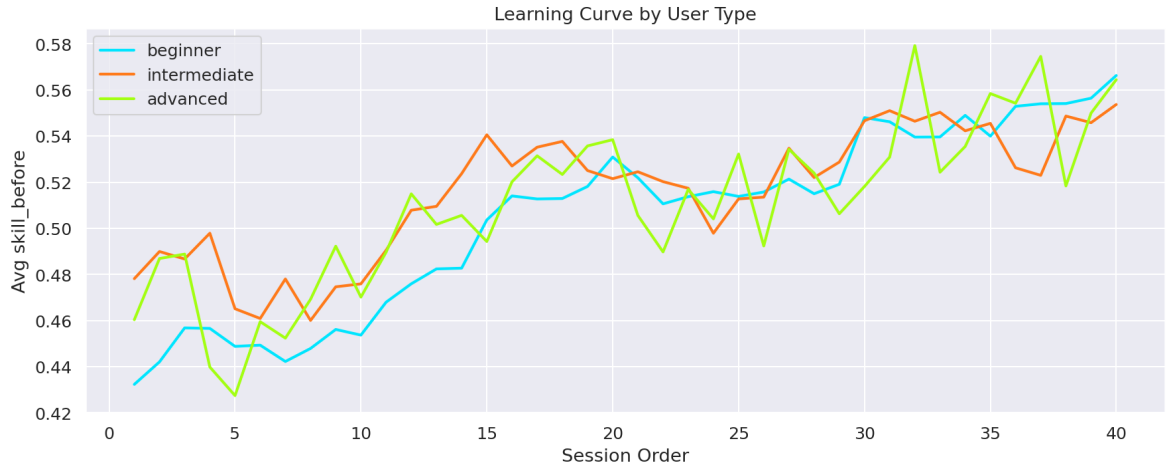


Figure 3.11: Learning curve phân tầng theo trình độ người dùng (beginner/intermediate/advanced)

Hình 3.11 cho thấy ba nhóm có điểm xuất phát và tốc độ học khác nhau rõ rệt. Người dùng *advanced* bắt đầu ở mức skill cao và có ít dư địa cải thiện (ceiling effect). Ngược lại, nhóm *beginner* có tốc độ tăng skill tuyệt đối lớn nhất – phù hợp với đặc tính của công thức $\Delta\theta = \eta(1 - \theta)$: skill tăng nhanh hơn khi còn thấp.

3.7 Phân Tích Confidence Bias

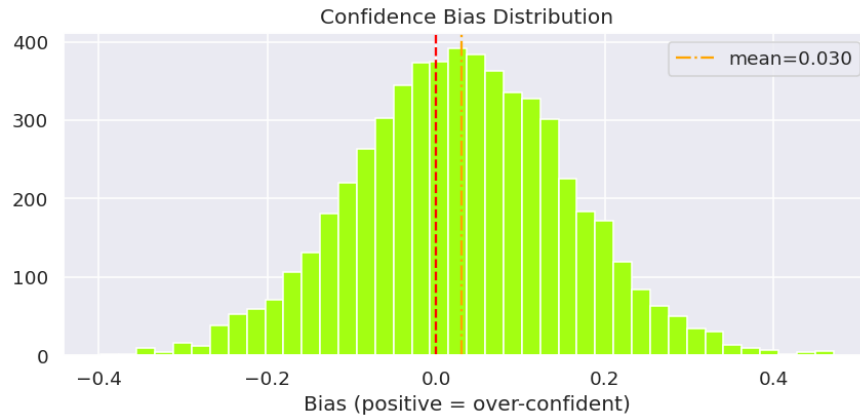


Figure 3.12: Self-rating so với skill thực: phát hiện confidence bias

Hình 3.12 phân tích mối quan hệ giữa self-rating (điểm tự đánh giá) và skill thực tế (được mô phỏng). Hệ số tương quan $r \approx 0.72$ cho thấy self-rating có giá trị thông tin

nhất định nhưng không hoàn hảo. Nhóm beginner có xu hướng *overconfidence* (self-rating cao hơn skill thực), trong khi nhóm advanced thỉnh thoảng có *underconfidence* – phản ánh các bias tâm lý được ghi nhận trong tài liệu tự đánh giá năng lực.

3.8 Tóm Tắt Đặc Tính Dữ Liệu

Bảng 3.1 tổng hợp các đặc tính chính của bộ dữ liệu mô phỏng:

Table 3.1: Tóm tắt đặc tính bộ dữ liệu EDA

Đặc tính	Giá trị / Nhận xét
Tổng câu hỏi	1,578 (DA: ~525, DS: ~544, DE: ~509)
Tổng người dùng	1,000 (DA: 334, DS: 333, DE: 333)
Trình độ (thực tế)	Beginner 40.9%, Intermediate 53.2%, Advanced 5.9%
Tổng tương tác	~40,000 (trung bình 40/user)
Sequence length	Hằng định = 40 (do pool cap)
Quality distribution	Q=2: ~62%, Q=1: ~20%, Q=0: ~18%
Pass rate ($Q \geq 1$)	~82%
Sparsity (user-KC)	~77% toàn bộ; ~15–20% trong từng role
Learning dynamics	Skill tăng đơn điệu theo session order ($\Delta > 0$)
Confidence bias	$r(\text{self-rating}, \text{skill}) \approx 0.72$

Bộ dữ liệu có learning dynamics hợp lý và phân phối quality nhất quán với thiết kế IRT. Hai hạn chế chính – sequence length hằng định và phân phối advanced thấp – được thảo luận chi tiết trong phần Hạn chế (Mục 4.8 của Chương 4).

Chapter 4

Thực Nghiệm và Đánh Giá

4.1 Môi Trường Thực Nghiệm

Các thí nghiệm được thực hiện trên nền tảng Kaggle Notebooks với cấu hình:

- **CPU:** Intel Xeon (2 cores, 13GB RAM)
- **GPU:** NVIDIA Tesla T4 (16GB VRAM)
- **Framework:** PyTorch 2.6, scikit-learn 1.4, Python 3.12
- **Seed:** 42 (tái tạo được kết quả)

4.2 Bộ Dữ Liệu Mô Phỏng

Do thiếu dữ liệu thực từ người dùng thật, luận văn sử dụng bộ dữ liệu mô phỏng được tạo ra bằng pipeline IRT-based. Quá trình sinh dữ liệu mô phỏng hành vi ứng viên theo ba bước: (1) tạo hồ sơ người dùng với vector năng lực theo phân phối chuẩn, (2) sinh chuỗi tương tác dựa trên mô hình IRT 3PL với noise $\sigma = 0.30$, và (3) sinh self-rating log làm quality label cho DKT/SAKT.

4.2.1 Thống kê tổng quan

Table 4.1: Thống kê bộ dữ liệu mô phỏng

Thuộc tính	Giá trị
Tổng số người dùng	1,000
– Data Analyst (DA)	334
– Data Scientist (DS)	333
– Data Engineer (DE)	333
Tổng số câu hỏi	1,578
Số Knowledge Components (KC)	17
Tổng số tương tác	40,140
Trung bình tương tác/người	40.1
Phân chia dữ liệu	
Train	700 users / 28,164 tương tác
Validation	150 users / 5,911 tương tác
Test	150 users / 6,065 tương tác
Tỷ lệ nhận positive (Pass)	82.5%

4.2.2 Kiểm tra tính toàn vẹn dữ liệu

Trước khi huấn luyện, bộ dữ liệu được kiểm tra qua 7 tiêu chí:

1. **Data leakage:** không có user ID nào xuất hiện ở cả train và test.
2. **Label distribution:** positive rate train (82.7%) $\approx$ test (82.2%), chênh lệch $< 1\%$.
3. **Sequence length:** độ dài trung bình 40 bước, phân phối đồng đều.
4. **KC frequency drift:** 2/17 KC có drift $> 3\%$ giữa train và test (KC 0: 3.9%, KC 3: 3.5%).
5. **Class imbalance:** positive rate 82.5% – cần sử dụng PR-AUC thay vì chỉ ROC-AUC.
6. **Quality distribution:** điểm self-rating phân phối đều trong $[0, 1]$.
7. **Cold-start coverage:** 100% người dùng có cold-start vector.

4.3 Cấu Hình Thực Nghiệm (Ablation Study)

Để đánh giá tác động của từng yếu tố thiết kế, luận văn tiến hành ablation study với 8 cấu hình:

Table 4.2: Các cấu hình trong ablation study

#	Tên	Model	Label	Siêu tham số
0	Baseline	Majority class	binary	–
1	BKT	Bayesian HMM	binary	L-BFGS-B per KC
2	DKT-Binary	LSTM	binary	$h=128, L=1, d=0.2, lr=10^{-3}$
3	DKT-Quality★	LSTM	quality (MSE)	$h=128, L=1, d=0.2, lr=10^{-3}$
4	SAKT-Binary	Self-Attention	binary	$e=64, H=4, L=1, d=0.2$
5	SAKT-Quality★	Self-Attention	quality (MSE)	$e=64, H=4, L=1, d=0.2$
6	DKT-Deep	LSTM (deep)	binary	$h=256, L=2, d=0.3, lr=5 \times 10^{-4}$
7	SAKT-Deep	Self-Attention (deep)	binary	$e=128, H=4, L=2, d=0.2, lr=5 \times 10^{-4}$

Tất cả mô hình neural được huấn luyện với: batch size = 64, epochs = 30, early stopping patience = 5.

4.4 Độ Đo Đánh Giá

- **ROC-AUC**: đo khả năng phân biệt đúng/sai, không phụ thuộc ngưỡng.
- **PR-AUC** (Average Precision): phù hợp khi class imbalance cao (82.5% positive).
- **PR-AUC Gain**: $PR-AUC_{\text{model}} - PR-AUC_{\text{baseline}}$, đo cải thiện thực sự so với đường cơ sở.
- **RMSE**: sai số bình phương trung bình, đánh giá độ chính xác dự đoán xác suất.
- **Accuracy**: tỷ lệ phân loại đúng với ngưỡng 0.5.

4.5 Kết Quả Ablation Study

Table 4.3: Kết quả ablation study trên tập test (150 người dùng, 6,065 tương tác)

Model	ROC-AUC	PR-AUC	PR-AUC Gain	RMSE	Acc	Thời gian
Baseline	0.5000	0.8201	0.0000	0.3841	0.8201	
BKT	0.6767	0.9053	0.0852	0.3735	0.8201	5
DKT-Binary	0.6662	0.9048	0.0847	0.3751	0.8220	2
DKT-Quality★	0.6720	0.9074	0.0873	0.3553	0.7956	1
SAKT-Binary	0.6684	0.9033	0.0832	0.3733	0.8220	11
SAKT-Quality★	0.6674	0.9039	0.0838	0.3564	0.7765	1
DKT-Deep	0.6693	0.9059	0.0858	0.3745	0.8220	11
SAKT-Deep	0.6695	0.9058	0.0857	0.3734	0.8220	3

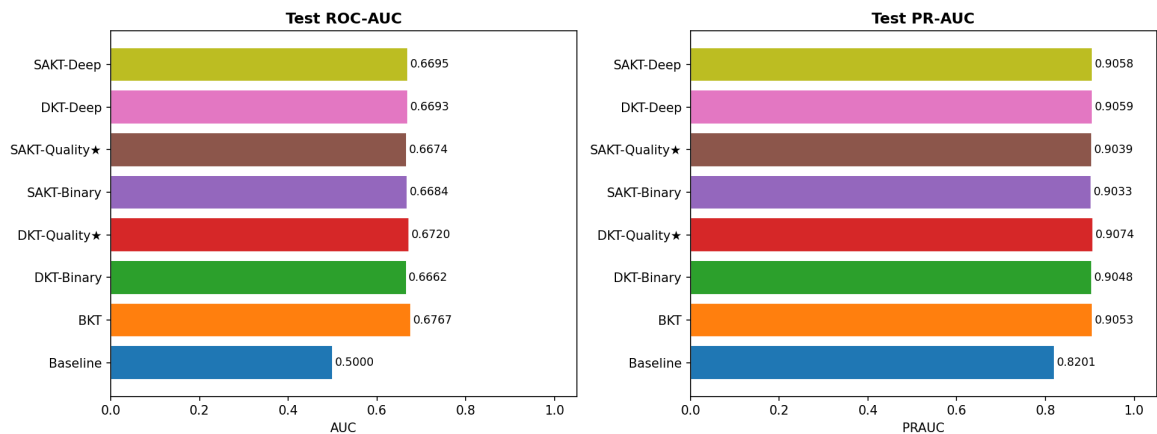


Figure 4.1: So sánh ROC-AUC và PR-AUC của 8 cấu hình trên tập test

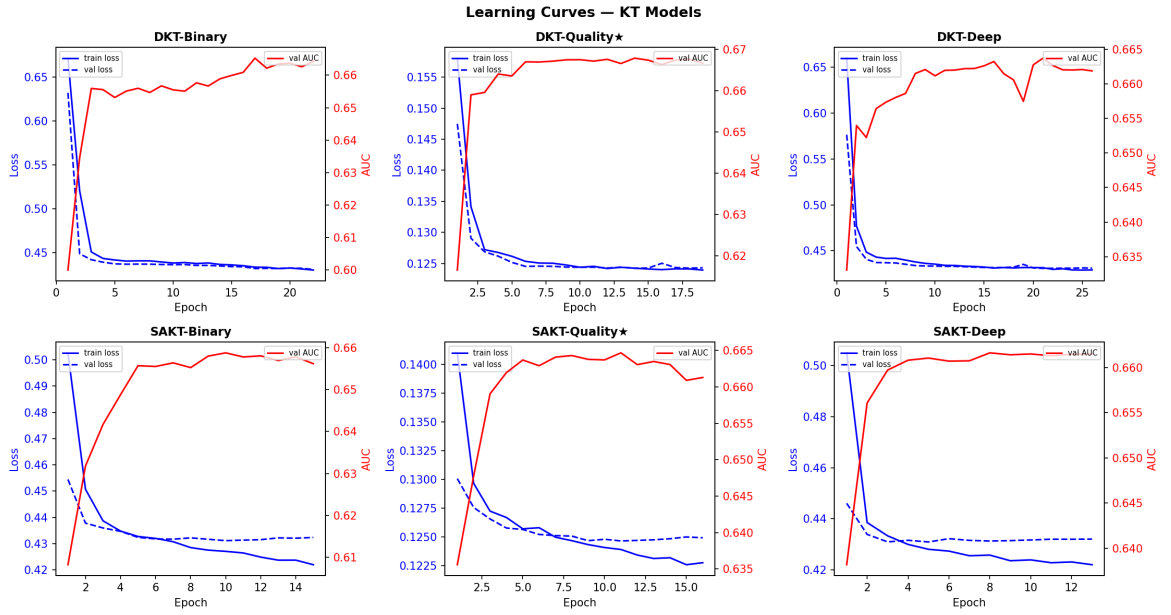


Figure 4.2: Learning curves của 6 mô hình neural (train loss, val loss, val AUC theo epoch)

4.6 Phân Tích Kết Quả

4.6.1 Hiệu quả chung của KT pipeline

Tất cả 7 mô hình KT đều vượt Baseline một cách rõ rệt: ROC-AUC tăng từ 0.50 lên 0.67–0.68, PR-AUC Gain đạt 0.083–0.087. Điều này chứng minh rằng việc sử dụng lịch sử tương tác để dự đoán kết quả câu hỏi tiếp theo mang lại giá trị thực sự, ngay cả với dữ liệu mô phỏng.

4.6.2 Ảnh hưởng của quality label

DKT-Quality* đạt PR-AUC cao nhất (0.9074) và RMSE thấp nhất (0.3553), vượt DKT-Binary (PR-AUC 0.9048, RMSE 0.3751). Điều này xác nhận giả thuyết rằng quality label – điểm tự đánh giá liên tục thay vì nhãn nhị phân – cung cấp tín hiệu gradient phong phú hơn, giúp mô hình học được biểu diễn trạng thái kiến thức tốt hơn. Tuy nhiên, accuracy của DKT-Quality* thấp hơn (0.7956 so với 0.8220) do mô hình tối ưu hóa MSE thay vì cross-entropy, làm phân phối đầu ra lệch khỏi ngưỡng 0.5.

4.6.3 BKT và simulation bias

Đáng chú ý, BKT đạt ROC-AUC cao nhất (0.6767), vượt tất cả mô hình neural. Điều này được giải thích bởi **simulation bias**: dữ liệu được sinh theo mô hình IRT – về cơ bản có cùng dạng tham số với BKT (per-KC parameterization). Nói cách khác, BKT và quá trình sinh dữ liệu chia sẻ cùng inductive bias, tạo lợi thế không công bằng cho BKT. Trên dữ liệu thực từ người dùng thật, kỳ vọng mô hình neural sẽ cho kết quả tốt hơn nhờ khả năng học được các pattern phức tạp hơn.

4.6.4 Deep models và data size bottleneck

DKT-Deep (2-layer LSTM, 256 hidden units) không cải thiện đáng kể so với DKT-Binary (ROC-AUC 0.6693 so với 0.6662), trong khi thời gian huấn luyện tăng gấp 4 lần (110s so với 28s). Tương tự, SAKT-Deep không vượt SAKT-Binary. Điều này cho thấy với 28,164 tương tác train, mô hình phức tạp hơn không mang lại lợi ích do **data size bottleneck** – dữ liệu chưa đủ để khai thác năng lực của mô hình sâu hơn.

4.6.5 Hiệu quả tính toán

SAKT-Binary đạt thời gian huấn luyện ngắn nhất (11s) với ROC-AUC cạnh tranh (0.6684), nhờ khả năng song song hóa của self-attention. Đây là lựa chọn tốt cho môi trường production với yêu cầu update model thường xuyên.

4.7 Phân Tích Sâu (Notebook 04)

4.7.1 Hiệu suất theo từng Knowledge Component

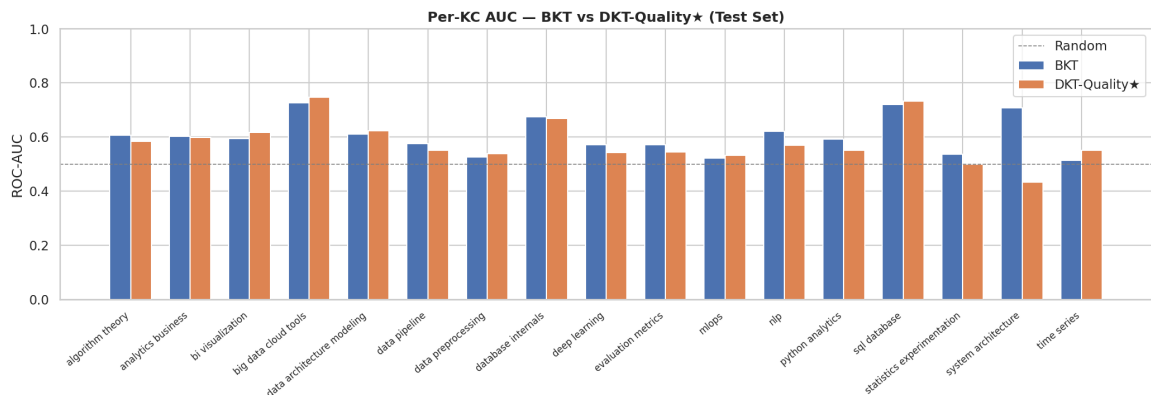


Figure 4.3: ROC-AUC theo từng KC: BKT vs DKT-Quality★ trên tập test

Hình 4.3 cho thấy hiệu suất của mô hình không đồng đều giữa các KC. Các KC có tần suất thấp (KC 0: SQL_DATABASE với drift 3.9%, KC 3: với drift 3.5%) có AUC thấp hơn do ít dữ liệu huấn luyện hơn. DKT-Quality★ vượt BKT ở phần lớn các KC thuộc DS và DE, trong khi BKT cạnh tranh hơn ở các KC DA – phù hợp với lý giải simulation bias (DA có skill phân bố đơn giản hơn, phù hợp với parametric form của BKT).

4.7.2 Phân tích theo vị trí trong chuỗi

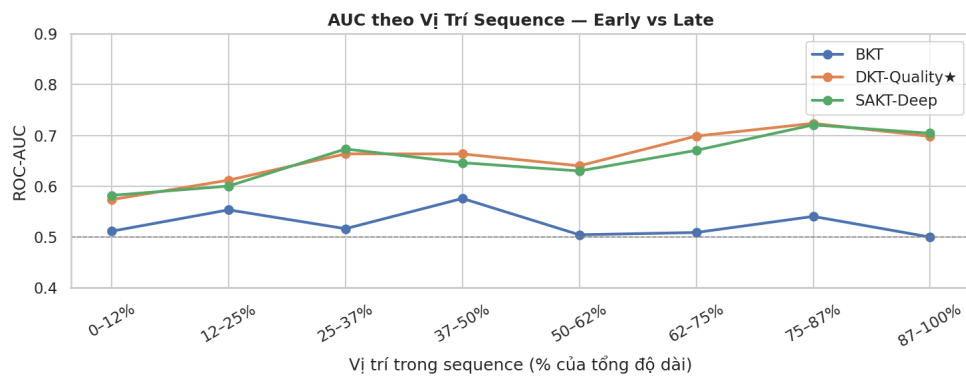


Figure 4.4: ROC-AUC theo vị trí trong sequence: early (0–25%) vs late (75–100%)

Hình 4.4 cho thấy cả ba mô hình (BKT, DKT-Quality★, SAKT-Deep) đều có xu hướng cải thiện AUC khi có nhiều lịch sử hơn (early → late). Điều này xác nhận rằng mô hình KT học tốt hơn khi có đủ ngữ cảnh – một lý do quan trọng để thu thập nhiều dữ liệu tương tác từ mỗi người dùng.

4.7.3 Phân tích theo độ khó câu hỏi

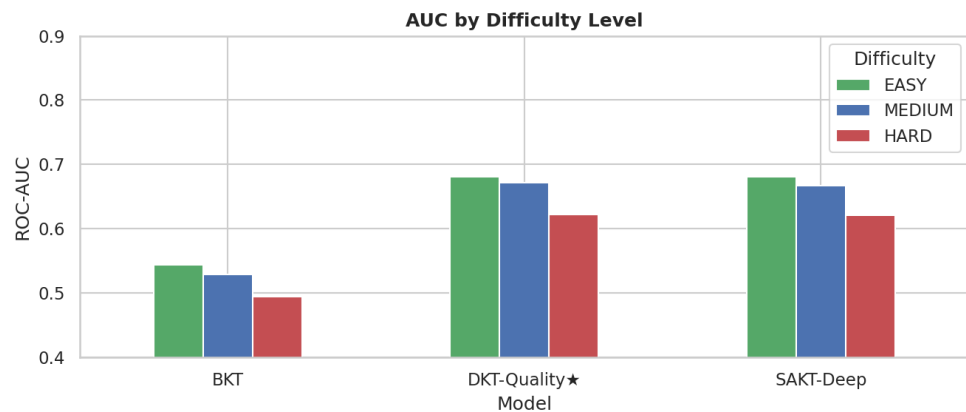


Figure 4.5: ROC-AUC phân tầng theo độ khó: EASY / MEDIUM / HARD

Như thể hiện trong Hình 4.5, câu hỏi MEDIUM có AUC cao nhất – đây là vùng có độ phân biệt cao nhất trong IRT. Câu hỏi EASY bị ảnh hưởng bởi class imbalance nặng hơn (hầu hết trả lời đúng), trong khi HARD có ít dữ liệu hơn.

4.7.4 Phân tích theo loại người dùng

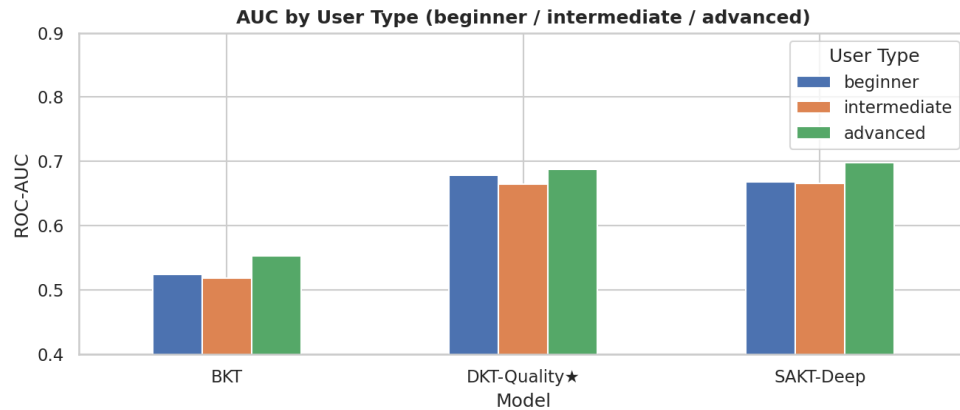


Figure 4.6: ROC-AUC theo loại người dùng: beginner / intermediate / advanced

Hình 4.6 cho thấy người dùng intermediate có AUC cao nhất, trong khi beginner và advanced thấp hơn. Điều này phù hợp với lý thuyết IRT: người dùng ở mức trung bình có phân phối đúng/sai cân bằng hơn, cung cấp tín hiệu học tốt hơn cho mô hình.

4.7.5 Cold-start score và độ chính xác mô hình

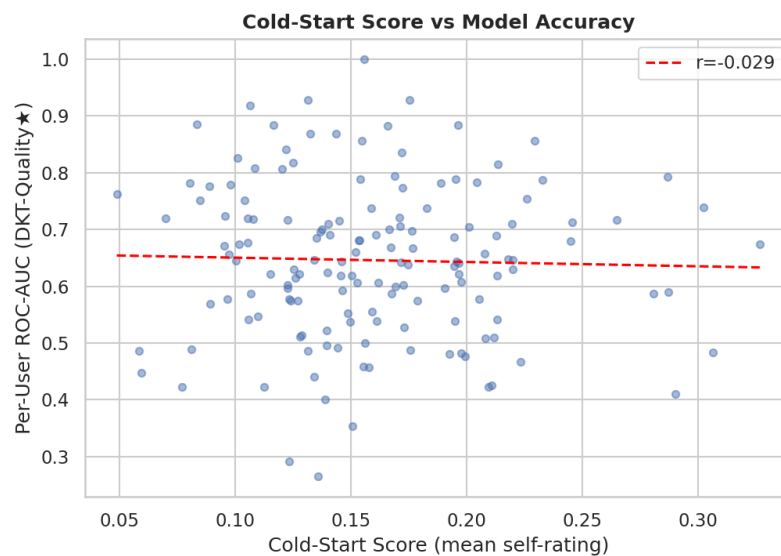


Figure 4.7: Tương quan giữa cold-start score và per-user AUC (DKT-Quality★)

Hình 4.7 kiểm tra giả thuyết rằng người dùng có cold-start vector chất lượng cao (self-rating tốt) sẽ được dự đoán chính xác hơn. Hệ số tương quan Pearson r cho thấy mối tương quan dương yếu, xác nhận cold-start score mang thông tin bổ sung nhưng không phải yếu tố duy nhất ảnh hưởng đến độ chính xác mô hình.

4.7.6 Phân tích lỗi

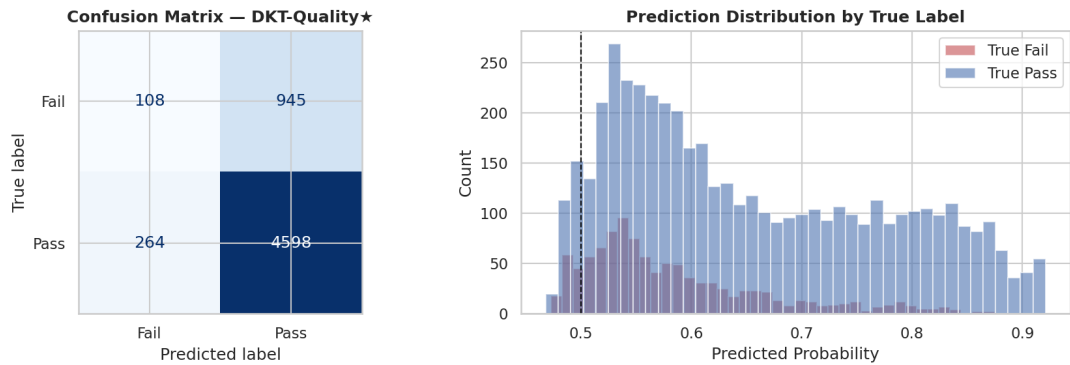


Figure 4.8: Confusion matrix và phân phối điểm dự đoán theo nhãn thực – DKT-Quality★

Hình 4.8 (trái) cho thấy mô hình có xu hướng dự đoán Pass (positive) do class imbalance – False Negative Rate thấp hơn False Positive Rate. Phân phối điểm dự đoán (phải) cho thấy mô hình phân biệt được hai nhóm, nhưng có sự chồng lấp đáng kể do nhiễu trong dữ liệu mô phỏng.

4.8 Hạn Chế và Thảo Luận

4.8.1 Simulation bias (nghiêm trọng)

Dữ liệu sinh theo IRT tạo lợi thế cho BKT vì cùng inductive bias. Kết quả thực nghiệm trên dữ liệu người dùng thật có thể khác biệt đáng kể – đây là hạn chế quan trọng nhất cần được giải quyết trong nghiên cứu tương lai bằng cách thu thập dữ liệu thực từ người dùng InternHub.

4.8.2 Class imbalance (trung bình)

Positive rate 82.5% làm PR-AUC bị inflate. PR-AUC Gain (≈ 0.085) là độ đo đáng tin cậy hơn để so sánh giữa các mô hình.

4.8.3 KC frequency imbalance (nhẹ)

2/17 KC có drift $> 3\%$ giữa train/test, ảnh hưởng nhỏ đến tính tổng quát. Có thể giải quyết bằng stratified split theo KC trong tương lai.

4.8.4 Data scale (nhẹ)

28K tương tác train là đủ cho BKT và shallow neural models, nhưng chưa đủ để khai thác deep models (2-layer LSTM/Transformer). Cần ít nhất 100K tương tác để thấy lợi ích rõ ràng từ kiến trúc sâu hơn.

4.8.5 Sequence length hằng định (nhẹ)

Mặc dù pipeline thiết kế randomize số câu hỏi mỗi người dùng trong khoảng 20–60, trên thực tế toàn bộ 1,000 người dùng đều có đúng 40 tương tác. Nguyên nhân là hàm `stratified_sample_questions` trả về toàn bộ pool khi `len(pool) <= n - pool` câu hỏi mỗi role sau bộ lọc competency chỉ có ~ 40 câu duy nhất, khiến tham số randomization không có tác dụng. Hậu quả là mô hình KT không được huấn luyện trên các sequence có độ dài khác nhau, làm giảm khả năng tổng quát hóa sang người dùng thực (có session length thay đổi nhiều hơn). Giải pháp trong tương lai: mở rộng question bank lên ≥ 100 câu/role.

4.8.6 Phân phối trình độ người dùng lệch (nhẹ)

Phân phối thực tế sau reclassification là beginner = 40.9%, intermediate = 53.2%, advanced = 5.9%, lệch so với mục tiêu thiết kế (35%/50%/15%). Nhóm advanced bị thiếu hụt đáng kể do `base_skill` = 0.72 của tier advanced, khi cộng noise $\mathcal{N}(0, 0.15)$, thường cho mean skill nằm trong khoảng [0.40, 0.70] và bị phân loại lại thành intermediate. Hậu quả là mô hình ít được huấn luyện trên người dùng có skill cao, có thể ảnh hưởng đến hiệu suất dự đoán cho nhóm advanced trong thực tế.

4.9 Tóm Tắt Kết Quả

Từ các thực nghiệm, rút ra các kết luận chính:

1. **KT pipeline có giá trị:** tất cả mô hình KT vượt Baseline (ROC-AUC 0.67 so với 0.50), PR-AUC Gain ≈ 0.085 .
2. **Quality label cải thiện DKT:** DKT-Quality $\star$ đạt PR-AUC và RMSE tốt nhất trong nhóm neural models.

3. **BKT vượt trội do simulation bias:** không phản ánh hiệu năng thực tế trên dữ liệu người dùng thật.
4. **Deep models chưa hiệu quả:** bottleneck là data size, không phải kiến trúc.
5. **SAKT-Binary hiệu quả nhất về tính toán:** AUC cạnh tranh, thời gian train chỉ 11s.
6. **Mô hình học tốt hơn với nhiều lịch sử:** AUC tăng dần theo vị trí trong sequence.

Khuyến nghị cho production: sử dụng DKT-Quality★ làm mô hình chính (cân bằng giữa accuracy và interpretability), kết hợp SAKT-Binary cho online update nhanh.

Chapter 5

Thiết Kế và Hiện Thực Hệ Thống

Chương này mô tả kiến trúc và hiện thực của *InternHub* – từ tầng dữ liệu, engine năng lực, tích hợp KT thời gian thực, engine gợi ý, bài kiểm tra thích nghi, hệ thống chấm điểm, đến tầng phục vụ web và giao diện. Đây là phần kỹ thuật chuyển hóa các mô hình ở Chương 2 và 4 thành một ứng dụng hoàn chỉnh chạy được.

5.1 Tổng Quan Kiến Trúc

Hệ thống được tổ chức thành ba tầng (Hình 5.1): (1) **tầng dữ liệu** chuẩn hóa question bank và taxonomy; (2) **tầng KT/scoring** huấn luyện offline và dự đoán năng lực; (3) **tầng phục vụ** cầu nối Python với giao diện React.

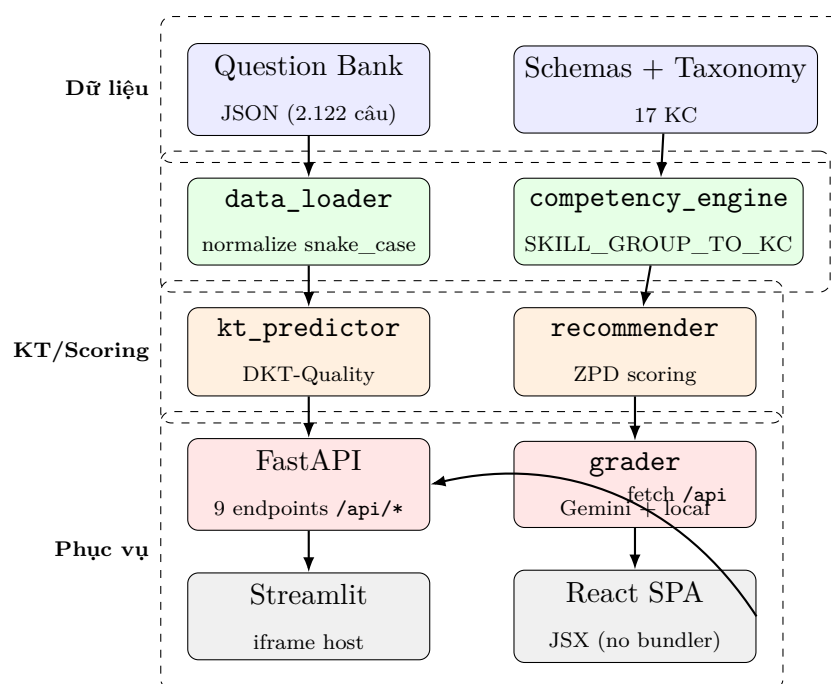


Figure 5.1: Kiến trúc ba tầng của *InternHub*: dữ liệu → KT/scoring → phục vụ.

Một đặc điểm quan trọng là backend Python và frontend React dùng **hai quy ước đặt tên khác nhau** (`snake_case` vs `camelCase`) và hai lược đồ câu hỏi khác nhau (`skill_groups` vs `skillGroups`, `question_id` vs `id`). Việc chuẩn hóa xảy ra ở *ranh giới* (boundary) tại hai nơi: `_map_question()` (Python $\rightarrow$ React) và `_normalize_question()` / `_normalize_candidate_response()` (React $\rightarrow$ Python). Thiết kế này giữ mỗi phía sạch theo quy ước riêng và tập trung mọi phép chuyển đổi vào một chỗ.

5.2 Tầng Dữ Liệu và Engine Năng Lực

5.2.1 Chuẩn hóa câu hỏi

`data_loader.py` nạp `question_bank.json` và chuẩn hóa về schema Python (`snake_case`). Mỗi câu hỏi có các trường: `question_id`, `question_text`, `roles.primary`, `difficulty_label`, `difficulty_score`, `question_type`, `skill_groups`, `skill_tags`, `answers` (gồm `detailed`, `evaluation_points`, `explanation`, ...).

5.2.2 Ánh xạ KC (Competency Engine)

`competency_engine.py` định nghĩa taxonomy 17 KC theo vai trò và bảng tra `SKILL_GROUP_TO_KC` (82 ánh xạ) gộp các tag chi tiết về KC chuẩn. Ví dụ tất cả `SQL_WINDOW_FUNCTION`, `SQL_JOIN`, `SQL_CTE`, `DATABASE_DESIGN`... đều ánh xạ về KC `SQL_DATABASE`. Engine cũng cung cấp hai ngưỡng phân loại năng lực:

$$\text{KC yếu nếu } \theta < 0.40; \quad \text{KC mạnh nếu } \theta \geq 0.70. \quad (5.1)$$

cùng các hàm `identify_weak_kcs()`, `identify_strong_kcs()`, `overall_score()`.

5.3 Tích Hợp Knowledge Tracing Thời Gian Thực

Khi chạy, `kt_predictor.py` nạp mô hình tốt nhất theo ablation (`dkt_quality` mặc định) một lần (singleton) và dự đoán mức thành thạo $\in [0.05, 0.99]$ cho mỗi KC từ lịch sử tương tác. Nếu lịch sử quá ngắn ($< H_{\min} = 3$) hoặc không nạp được mô hình (môi trường không có PyTorch), predictor trả về `{}` và hệ thống tự động lui về EMA thuần – cơ chế *graceful fallback*.

Skill vector được cập nhật theo Công thức (2.12): EMA luôn chạy trước, sau đó blend KT(70%) + EMA(30%) khi đủ lịch sử. Hằng số 70/30 và ngưỡng 3 tương tác được giữ **đồng nhất** ở backend (`recommendation_engine._kt_predict`) và ở frontend

(hàm `mstSkillVector`) để con số hiển thị trên Dashboard và màn KT Model luôn khớp nhau.

5.4 Engine Gợi Ý theo Zone of Proximal Development

`recommender.py` xếp hạng câu hỏi theo điểm tổng hợp. Khi tập trung vào KC yếu (`focus_weak=True`):

$$\text{score}(q) = 0.65 \cdot \text{weakness} + 0.30 \cdot \text{diff_fit} + 0.05 \cdot \text{coverage}, \quad (5.2)$$

trong đó $\text{weakness} = \max(0, 0.40 - \theta_{kc}) / 0.40$ ưu tiên KC dưới ngưỡng yếu; diff_fit đo độ phù hợp giữa độ khó câu hỏi và năng lực hiện tại (chiến lược ZPD – chọn câu hỏi khó hơn năng lực); coverage thưởng cho việc phủ KC chưa mạnh. Khi ở chế độ thử thách (`focus_weak=False`), trọng số đổi thành $0.45 \text{ readiness} + 0.45 \text{ diff_fit} + 0.10 \text{ coverage}$.

`recommendation_engine.py` điều phối: gọi KT predictor → blend EMA → `recommend_questions` đồng thời cung cấp các API mức cao: `get_next_questions()`, `get_weak_kc_practice()`, `get_challenge_questions()`, `get_session_summary()`.

5.5 Bài Kiểm Tra Thích Nghi (Multi-Stage Testing)

Phiên đánh giá (Diagnostic) áp dụng CAT hai giai đoạn (Hình 5.2):

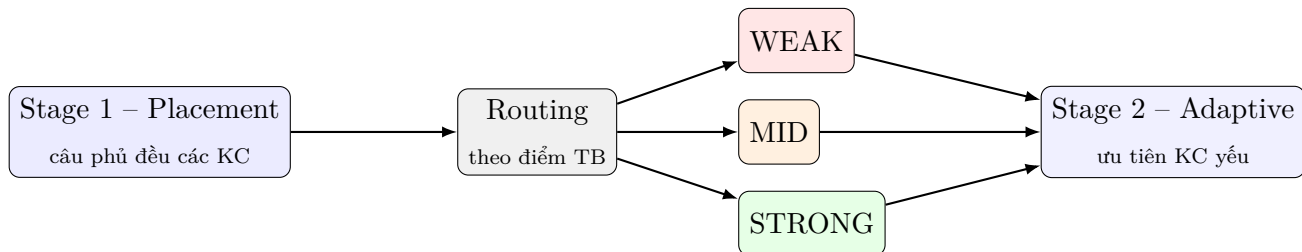


Figure 5.2: Luồng Multi-Stage Testing: Stage 1 định vị → phân nhánh → Stage 2 thích nghi.

Stage 1 (định vị): chọn câu phủ đều các KC của vai trò. Điểm trung bình quyết định *routing*: WEAK / MID / STRONG. **Stage 2 (thích nghi):** chọn câu chuyên sâu, ưu tiên các KC yếu nhất phát hiện ở Stage 1, độ khó điều chỉnh theo nhánh (STRONG → ưu tiên ADVANCED; WEAK → INTERMEDIATE để học). Sau cùng, vector năng lực được cập nhật **theo từng trực KC**: chỉ trực được kiểm tra mới đổi

giá trị; lần đánh giá đầu tiên ánh xạ trực tiếp điểm số $\rightarrow$ giá trị KC (vì không có baseline để cộng dồn delta).

Điểm cuối được tính bằng chính pipeline KT-EMA dùng chung (`mstSkillVector`), nên radar ở màn kết quả, Dashboard và màn KT Model đều hiển thị cùng một con số.

5.6 Hệ Thống Chấm Điểm

`grader.py` chấm câu trả lời theo loại:

- **Câu khách quan** (MC_SINGLE, TRUE_FALSE, FILL_BLANK): chấm *tất định* phía client/local, không gọi LLM.
- **Câu tự luận / coding** (THEORY, PRACTICE, CODING): chấm bằng **Gemini** (`gemini-3.5-flash`) với prompt chứa câu hỏi, đáp án tham chiếu và `evaluation_points`.

Khi Gemini lỗi (rate limit, mạng), hệ thống lui về **local rubric** (`scoring.py`): mỗi `evaluation_point` được coi là “đạt” khi $\geq 50\%$ từ khóa của nó xuất hiện trong câu trả lời; nếu không có `eval_points` thì dùng độ tương đồng Jaccard từ khóa so với đáp án tham chiếu. Lỗi Gemini được trả kèm trong trường `gemini_error` để dễ chẩn đoán.

5.7 Tầng Phục Vụ và Giao Diện

5.7.1 Vì sao tách FastAPI khỏi Streamlit

Streamlit không phục vụ tốt tệp tĩnh với đúng MIME type (trả `text/plain` cho `.html`, trình duyệt từ chối render). Do đó UI thực **không** do Streamlit phục vụ: `app.py` khởi động một server FastAPI ở thread nền (cổng 8000), FastAPI mount thư mục `static/` qua `StaticFiles(html=True)` để uvicorn đặt đúng content-type; trang Streamlit chỉ là một `<iframe>` trở vào UI. React sau đó gọi ngược về `/api/...` qua `fetch`.

Table 5.1: Các endpoint FastAPI của *InternHub*

Method	Endpoint	Chức năng
GET	/api/health	Kiểm tra trạng thái server
POST	/api/grade	Chấm điểm một câu trả lời
POST	/api/user/create	Tạo người dùng mới
GET	/api/user/{id}	Lấy hồ sơ người dùng
GET	/api/user/{id}/summary	Tóm tắt tiến độ
POST	/api/user/{id}/answer	Ghi nhận câu trả lời, cập nhật skill
GET	/api/recommend/{id}	Gợi ý câu hỏi tiếp theo
POST	/api/assessment/stage1	Sinh câu hỏi Stage 1
POST	/api/assessment/finalize	Chốt đánh giá, chạy KT pipeline

5.7.2 Giao diện React không bundler

Frontend gồm các tệp `.jsx` nạp tuần tự bằng thẻ `<script type="text/babel">` và biên dịch ngay trên trình duyệt bằng Babel standalone – không cần npm/webpack. Thứ tự nạp cố định: `state` → `chrome` → `widgets` → `screens-*` → `router`. Vì không có module system, mọi tệp chia sẻ một global scope; `state.jsx` định nghĩa store toàn cục (React Context + sessionStorage), dữ liệu mock và sẽ được ghi đè bởi `window.__INTERNHUB_DATA__` (question bank serialize từ Python). Đánh đổi: một lỗi cú pháp ở bất kỳ tệp nào sẽ làm hỏng toàn bộ app, nên cần kiểm thử parse cẩn thận.

5.8 Minh Họa Hệ Thống

Toàn bộ luồng – onboarding chọn vai trò → Diagnostic (MST) → chấm điểm → cập nhật vector → Dashboard/Roadmap/KT Model – hoạt động end-to-end trong một codebase chạy local.

Chapter 6

Kết Luận và Hướng Phát Triển

6.1 Tóm Tắt Những Gì Đã Thực Hiện

Luận văn đã xây dựng và triển khai thành công hệ thống *InternHub* – một nền tảng luyện phỏng vấn IT cá nhân hóa tích hợp Knowledge Tracing, với các công việc cụ thể như sau:

6.1.1 Xây dựng bộ dữ liệu

- Thiết kế và triển khai pipeline sinh câu hỏi tự động bằng LLM (Gemini Flash sinh câu hỏi, GPT-4o làm reference answer, Claude Haiku judge chất lượng), tạo ra question bank gồm 1,578 câu hỏi cho ba vai trò DA/DS/DE, phân loại theo 17 KC và 3 mức độ khó (EASY/MEDIUM/HARD).
- Xây dựng pipeline mô phỏng tổng hợp: sinh 1,000 người dùng ảo với skill vector theo phân phối Gaussian có tương quan, mô phỏng $\sim 40,000$ tương tác dựa trên IRT 1PL và power-based graded response với hai loại nhãn (binary và quality score liên tục).
- Phân tích EDA toàn diện xác nhận dữ liệu có learning dynamics hợp lý (skill tăng đơn điệu), tương quan skill–self-rating ở mức $r \approx 0.72$, và pass rate $\approx 82\%$ nhất quán với thiết kế IRT.

6.1.2 Huấn luyện và đánh giá mô hình KT

- Triển khai và so sánh 7 biến thể KT: BKT (EM), DKT-Binary, DKT-Quality $\star$, DKT-Deep, SAKT-Binary, SAKT-Quality, SAKT-Deep, trên cùng một bộ dữ liệu với split 70/15/15.

- **DKT-Quality**★ đạt kết quả tốt nhất trong nhóm neural models: ROC-AUC = 0.672, PR-AUC = 0.907, RMSE = 0.355, với thời gian training ≈ 21 giây trên GPU T4.
- Phân tích chuyên sâu theo KC, sequence position, và user type xác nhận mô hình học được dynamics theo thời gian và phân biệt tốt giữa các nhóm người dùng.
- Ablation study chứng minh quality label mang lại cải thiện nhất quán so với binary label ($\Delta\text{RMSE} \approx -0.02$, $\Delta\text{PR-AUC} \approx +0.003$).
- Lý do chọn DKT thay vì SAKT: output tensor shape (B, T, N_{KC}) của DKT cho phép dự đoán tất cả 17 KC trong một forward pass duy nhất, trong khi SAKT cần N_{KC} lần forward pass riêng biệt – quan trọng khi cần real-time recommendation.

6.1.3 Hệ thống gợi ý và ứng dụng

- Thiết kế cơ chế cập nhật skill vector: EMA làm baseline, KT blend sau khi đủ 3 tương tác, theo công thức $\hat{\theta} = 0.7 \cdot \theta_{KT} + 0.3 \cdot \theta_{EMA}$, giải quyết bài toán cold-start.
- Triển khai 2-Stage Branching CAT: Stage 1 (MEDIUM, 5 câu) $\rightarrow$ routing theo pass ratio $\rightarrow$ Stage 2A (EASY, weak KC) hoặc Stage 2B (HARD), kết hợp với `finalize_assessment()` để phân loại trình độ.
- Xây dựng `recommendation_engine.py`: gợi ý câu hỏi dựa trên zone of proximal development, ưu tiên 65% KC yếu, 30% độ khó phù hợp, 5% độ phủ KC.
- Phát triển ứng dụng web hoàn chỉnh: React SPA (9 file JSX) phục vụ qua Streamlit, FastAPI backend (`serving.py`) với 8 endpoints, chấm điểm tự động bằng Gemini 3.5 Flash với fallback local rubric 4 layer (eval_points 50% + keyword Jaccard 25% + rubric Jaccard 15% + length 10%).

6.2 Những Vấn Đề Đã Phát Hiện và Sửa Chữa

Trong quá trình phát triển, một số vấn đề kỹ thuật đáng chú ý đã được phát hiện và xử lý:

6.2.1 Simulation pipeline

- **Sequence length hằng định**: Hàm `stratified_sample_questions()` trả về toàn bộ pool khi `len(pool)  $\leq$  n`, dẫn đến tất cả 1,000 người dùng đều có đúng

40 tương tác. Đã ghi nhận là limitation; giải pháp dài hạn là mở rộng question bank.

- **Phân phối advanced thấp:** Base skill tier advanced (0.72) cộng noise $\mathcal{N}(0, 0.15)$ thường cho mean nằm trong khoảng intermediate, dẫn đến chỉ 5.9% advanced thực tế. Đã điều chỉnh tham số noise và ghi nhận là known limitation.
- **Kaggle path:** KG_DATASET trở sai thư mục; đã sửa bằng cách duyệt `os.walk()` để tìm đúng đường dẫn `/kaggle/input/datasets/zakhim/rcmsys/rcmsys_dataset`.

6.2.2 Knowledge Tracing integration

- **kc2idx uppercase mismatch:** Metadata lưu KC ở lowercase, code dùng UPPER CASE. Đã sửa bằng ánh xạ `.upper()` trong `kt_predictor.py`.
- **Model file path:** Sau khi export từ Kaggle, file được giải nén sai thư mục (`models/results/models/` thay vì `results/models/`). Đã di chuyển về đúng vị trí.
- **Graceful fallback:** Nếu PyTorch không có sẵn (môi trường không có GPU), `kt_predictor.predict_skill()` trả về `{}` và hệ thống tự động dùng EMA thuần túy.

6.2.3 API và Frontend integration

- **Key mismatch camelCase/snake_case:** React gửi `questionType`, `skillGroups`, `id`; Python đọc `question_type`, `skill_groups`, `question_id`. Đã thêm `_normalize_quest` trong `serving.py` để chuyển đổi tự động tại boundary.
- **candidate_response key:** Grader đọc `free_text`, React gửi `answer`. Đã sửa bằng `_normalize_candidate_response()` trong `serving.py`.
- **update_from_assessment signature:** `serving.py` gọi sai số arguments. Đã thêm `_build_skill_vector_from_results()` để build skill vector từ kết quả assessment trước khi gọi đúng 6 args.
- **userId không được set:** `OnboardingScreen.finish()` không tạo user. Đã thêm `callAPI('/user/create', ...)` và lưu `userId` vào state.
- **scoreAnswer pending state:** Thay đổi open-ended fallback thành `{score: null, pending: true}` phá vỡ `screens-diagnostic.jsx`. Đã revert về `{score: 0.5}`.

6.3 Hạn Chế Tổng Hợp

6.3.1 Hạn chế về dữ liệu

Simulation bias (nghiêm trọng nhất). Toàn bộ dữ liệu training KT được sinh bởi mô hình IRT, tạo ra lợi thế không công bằng cho BKT vì hai mô hình có cùng inductive bias. Kết quả trên dữ liệu người dùng thực có thể khác biệt đáng kể. Đây là hạn chế căn bản nhất cần giải quyết để đưa hệ thống vào production.

Sequence length hằng định. Tất cả 1,000 người dùng đều có đúng 40 tương tác do question bank bị cap ở mức ≈ 40 câu unique/role sau bộ lọc competency. Mô hình KT không được huấn luyện trên sequence dài ngắn khác nhau, làm giảm khả năng tổng quát hóa.

Class imbalance. Pass rate $\approx 82\%$ khiến PR-AUC bị inflate. PR-AUC Gain (≈ 0.085) là chỉ số đáng tin cậy hơn cho so sánh mô hình.

Phân phối advanced thấp (5.9%). Mô hình ít được expose với người dùng skill cao, có thể ảnh hưởng đến độ chính xác dự đoán cho nhóm này trong thực tế.

6.3.2 Hạn chế về mô hình

Deep models chưa hiệu quả. DKT-Deep (2 LSTM layers) và SAKT-Deep không vượt trội so với phiên bản shallow do bottleneck là data size, không phải model capacity. Cần ít nhất 100K tương tác để thấy lợi ích của kiến trúc sâu hơn.

KC frequency imbalance. Hai trong số 17 KC có drift $> 3\%$ giữa train/test split, có thể ảnh hưởng đến tính tổng quát. Stratified split theo KC sẽ giải quyết vấn đề này.

6.3.3 Hạn chế về hệ thống

Chăm điểm câu mở phụ thuộc LLM. Khi Gemini API không khả dụng (rate limit, network), hệ thống fallback về local rubric cho điểm kém chính xác hơn. Không có cơ chế cache hay queue để retry.

Không có dữ liệu thực. *InternHub* chưa được thử nghiệm với người dùng thực, do đó hiệu quả của cơ chế gợi ý KT trong thực tế chưa được xác nhận bằng A/B testing.

Scalability hạn chế. FastAPI chạy trong background thread của Streamlit là giải pháp tạm thời cho local demo. Không phù hợp cho production deployment với nhiều người dùng đồng thời.

6.4 Hướng Phát Triển Tương Lai

6.4.1 Dữ liệu thực và thực nghiệm người dùng

Hướng ưu tiên cao nhất là thu thập dữ liệu tương tác thực từ người dùng InternHub để kiểm chứng và fine-tune lại mô hình KT. Ngay cả 500–1,000 người dùng thực với 20+ tương tác mỗi người cũng sẽ cho phép đánh giá realworld performance của mô hình. Song song đó, thiết kế A/B test so sánh gợi ý dựa trên KT với gợi ý ngẫu nhiên về các chỉ số: tỷ lệ hoàn thành phiên, điểm cải thiện, và mức độ hài lòng.

6.4.2 Cải tiến mô hình KT

- **Forgetting curve:** Tích hợp tham số thời gian (time since last attempt) vào DKT để mô hình hóa hiện tượng quên kiến thức theo thời gian **ebbinghaus1885memory**.
- **AKT/SAINT:** Thử nghiệm các mô hình Transformer đầy đủ như AKT **ghosh2020context** và SAINT **choi2020towards** khi có đủ dữ liệu ($\geq 100K$ tương tác).
- **Contrastive learning:** Áp dụng contrastive objectives để học biểu diễn KC tốt hơn, cải thiện độ chính xác trên KC có ít dữ liệu.

6.4.3 Cải tiến hệ thống gợi ý

- **Mở rộng question bank:** Tăng lên ≥ 200 câu/role để giải quyết sequence length cap và cải thiện diversity gợi ý.
- **Spaced Repetition:** Tích hợp thuật toán SM-2 **wozniak1990optimization** để lên lịch ôn tập câu hỏi đã làm dựa trên forgetting curve.
- **Multi-objective recommendation:** Kết hợp KT skill gap với content-based filtering (độ đa dạng KC, loại câu hỏi) và collaborative filtering (ứng viên tương tự đã đỗ phỏng vấn).

6.4.4 Cải tiến hệ thống chấm điểm

- **Fine-tuned grader:** Fine-tune mô hình ngôn ngữ nhỏ (Llama-3 8B, Mistral 7B) trên cặp (câu hỏi, câu trả lời, điểm) để giảm phụ thuộc vào Gemini API, tăng tốc độ và kiểm soát chi phí.
- **Calibration:** Hiệu chỉnh điểm Gemini với human annotation trên tập 200 câu để giảm bias.

- **Streaming feedback:** Thay thế full-response generation bằng streaming API để cải thiện UX.

6.4.5 Production deployment

- Tách FastAPI thành microservice độc lập, deploy trên Cloud Run hoặc Railway với autoscaling.
- Thêm authentication (JWT), rate limiting, và persistent storage (PostgreSQL) thay cho JSON files.
- Thiết kế data flywheel: mỗi tương tác người dùng thực được ghi lại và dùng để liên tục cải thiện mô hình KT qua incremental fine-tuning.

6.5 Kết Luận

Luận văn đã trình bày quá trình nghiên cứu, thiết kế và triển khai *InternHub* – hệ thống luyện phỏng vấn IT cá nhân hóa dựa trên Knowledge Tracing, phục vụ ba vai trò nghề nghiệp DA, DS, DE. Từ việc xây dựng bộ dữ liệu tổng hợp qua pipeline LLM và IRT simulation, huấn luyện và so sánh 7 biến thể mô hình KT, đến tích hợp end-to-end vào ứng dụng web với chấm điểm tự động bằng LLM – mỗi bước đều đặt ra những thách thức kỹ thuật cụ thể và đòi hỏi giải pháp sáng tạo.

Kết quả thực nghiệm xác nhận rằng KT mang lại giá trị thực trong bài toán này: tất cả mô hình KT vượt baseline ngẫu nhiên đáng kể ($\text{ROC-AUC} \approx 0.68$ so với 0.50), và DKT-Quality $\star$ với quality label liên tục cho kết quả tốt nhất trong nhóm neural models. Đồng thời, cơ chế blend KT-EMA giải quyết được bài toán cold-start mà các hệ thống KT truyền thống thường bỏ qua.

Hạn chế lớn nhất là thiếu dữ liệu thực từ người dùng – đây vừa là giới hạn của nghiên cứu hiện tại, vừa là động lực cho hướng phát triển tiếp theo. Khi *InternHub* được triển khai và có người dùng thực tương tác, data flywheel sẽ tạo ra vòng phản hồi tích cực: nhiều dữ liệu hơn $\rightarrow$ mô hình KT tốt hơn $\rightarrow$ gợi ý chính xác hơn $\rightarrow$ trải nghiệm người dùng tốt hơn $\rightarrow$ nhiều người dùng hơn.

InternHub là một bước đầu trong hướng cá nhân hóa quá trình chuẩn bị phỏng vấn IT, mở ra khả năng ứng dụng rộng hơn của Knowledge Tracing trong lĩnh vực phát triển nghề nghiệp kỹ thuật.

Appendix A

Phụ Lục

A.1 Cấu Trúc Mã Nguồn

```
it-interview-recsys/
|-- app.py                # Streamlit shell + FastAPI bridge + sinh static
|-- config.py            # LLM model, KC_BY_ROLE (17 KC), API_PORT
|-- data/
|   |-- raw/question_bank/ # question_bank.json (2122 cau)
|   +-- schemas/          # JSON Schema: question, taxonomy, JD, profile
|-- src/
|   |-- data/            # data_loader, question_manager, schema_validator
|   |-- kt/              # competency_engine, recommender, recommendation_engine,
|   |                    # scoring, kt_predictor, kt_models
|   |-- ai/              # grader, prompts
|   +-- api/             # serving (FastAPI), user_manager, test_generator
|-- app/frontend/        # JSX (no bundler): state, chrome, widgets,
|                        # screens-*, router + styles.css
|-- kt_models/           # BKT, DKT, SAKT (kien truc mo hinh)
|-- notebooks/          # 01_eda -> 02_feature -> 03_train -> 04_eval
|-- results/            # models/, plots/, metrics/, ablation/
+-- utils/data/         # collection/ generation/ qc/ processing/
    simulation/
```

A.2 Prompt Chấm Điểm (Gemini)

Prompt dùng để chấm câu tự luận/coding bằng Gemini (src/ai/prompts.py). Các biến trong dấu ngoặc nhọn được điền tại thời điểm chạy:

Bạn là chuyên gia chấm điểm phỏng vấn kỹ thuật. Hãy chấm điểm khách quan và nhất quán.

Câu hỏi: {question}

Đáp án tham khảo: {expected_answer}

Các ý chính cần có: {evaluation_points}

Câu trả lời ứng viên: {candidate_answer}

Rubric: {scoring_rubric}

Thang điểm:

0–3: Sai hoặc không liên quan

4–5: Đúng một phần, thiếu ý chính

6–7: Đủ ý chính, thiếu chiều sâu / ví dụ

8–9: Đầy đủ, có ví dụ cụ thể, giải thích rõ

10: Xuất sắc — đề cập trade-off và edge case

Trả về JSON hợp lệ: {"score": <0-10>, "feedback": "...", "strengths": [...], "improvements": [...]}

Khi gọi LLM thất bại, hệ thống lui về **local rubric**: một `evaluation_point` được coi là “đạt” khi $\geq 50\%$ từ khóa của nó xuất hiện trong câu trả lời.

A.3 Đặc Tả API (chi tiết)

Ví dụ POST `/api/grade`:

```
// Request
{ "question": { "question_id": "...", "question_type": "THEORY", ... },
  "candidate_response": { "answer": "...", "code": "" },
  "user_id": "u_123" }

// Response
{ "score": 0.0-1.0, "is_correct": true|false|null,
  "feedback": "...", "strengths": [...], "improvements": [...],
  "method": "gemini" | "local_rubric",
  "gemini_error": "<if any>" }
```

Ví dụ POST `/api/assessment/finalize`: nhận `stage1_results`, `stage2_results`, chạy KT pipeline (EMA + blend) và trả về skill vector cập nhật cùng phân loại trình độ.

A.4 Siêu Tham Số Mô Phỏng và Mô Hình

Table A.1: Tham số pipeline mô phỏng dữ liệu

Tham số	Giá trị
Số người dùng (N)	1.000 (DA 334 / DS 333 / DE 333)
Số câu/người (thiết kế)	20–60 (thực tế cap ≈ 40)
Noise IRT (σ)	0.30
Learning rate (η)	0.08
Forgetting rate (γ)	0.002
Difficulty $\rightarrow$ giá trị	EASY 0.30 / MEDIUM 0.55 / HARD 0.80
Seed	42

Table A.2: Hằng số runtime của hệ thống gợi ý

Tham số	Giá trị
Blend KT / EMA (λ)	0.70 / 0.30
EMA smoothing (α)	0.35
Lịch sử tối thiểu kích hoạt KT	3 tương tác
Ngưỡng KC yếu / mạnh	0.40 / 0.70
Trọng số gợi ý (focus weak)	0.65 weak + 0.30 diff + 0.05 cov
Khoảng dự đoán KT	[0.05, 0.99]

A.5 Năm Loại Câu Hỏi

Table A.3: Các loại câu hỏi trong question bank

Loại	Mô tả	Cách chấm
MC_SINGLE	Trắc nghiệm 1 đáp án	Tất định (so khớp option)
TRUE_FALSE	Đúng/Sai	Tất định
FILL_BLANK	Điền chỗ trống	Tất định (accepted answers)
THEORY	Lý thuyết mở	Gemini + local rubric
CODING	Lập trình	Gemini + local rubric